Software Factory Foundations with OpenShift Dev Spaces

Version 1, 2026-10-26

Software Factory & Developer Experience

Introduction

Welcome to this quick course on the ***Software Factory Foundations with OpenShift Dev Spaces***. This course is the `second` in a series of `six` courses about **Application Platform Delivery**:

- *Introduction to Application Platforms and Platform Engineering*
- *Software Factory Foundations with OpenShift Dev Spaces* (*This course*)
- *Software Delivery with OpenShift Pipelines and GitOps*
- *DevEx with Red Hat Developer Hub*
- *Application Modernization with MTA*
- *Trusted Software Supply Chain (TSSC)* (Coming Soon)

What's in this course?

In this module, you'll learn how an Application Platform enables a **Software Factory** and why **Git** is fundamental to modern application delivery. You'll prepare the required platform services, configure a cloud-native development environment using **Red Hat OpenShift Dev Spaces** and **GitLab**, and deploy the **ETX App Base application**.

By the end of this module, you'll have a fully configured development environment and a running application that you'll use throughout the remainder of this learning path.

Duration: 3.00 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- Andrew Block (SME)
- Bill Wheatley (SME)
- Bryan Saunders (SME)
- Evan Shortiss (SME)
- Julio Villarreal Pelegrino (SME)
- Kristopher Godek (SME)
- Roman Martin Gil (SME)
- Senthil Kumar (SME)
- Sergio Canales Espinoza (SME)
- Yogita Soni (Content Architect)
- Lisa Edgell (Learning Product Manager)

- Caitlyn Rymarchyk (Learning Product Manager)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

- Explain how an Application Platform enables a **Software Factory** for standardized software delivery.
- Describe the role of Git in collaborative development, CI/CD pipelines, and GitOps workflows.
- Configure **Red Hat OpenShift Dev Spaces** and integrate it with **GitLab**.
- Create and use a reusable Devfile to provision cloud-native development workspaces.
- Launch a browser-based development environment and connect it to the application platform.
- Deploy the **ETX App Base application** and verify that it is running successfully.

Prerequisites

This course assumes that you have the following prior experience:

1. Red Hat OpenShift Technical Overview (DO080) v4.20 or Kubernetes Basics

Lab Environment

Before starting the hands-on exercises, provision the required lab environments from the Red Hat Demo Platform (RHDP).

This learning path uses two OpenShift clusters that represent different stages of a software delivery pipeline. Both environments should be provisioned before you begin the labs.



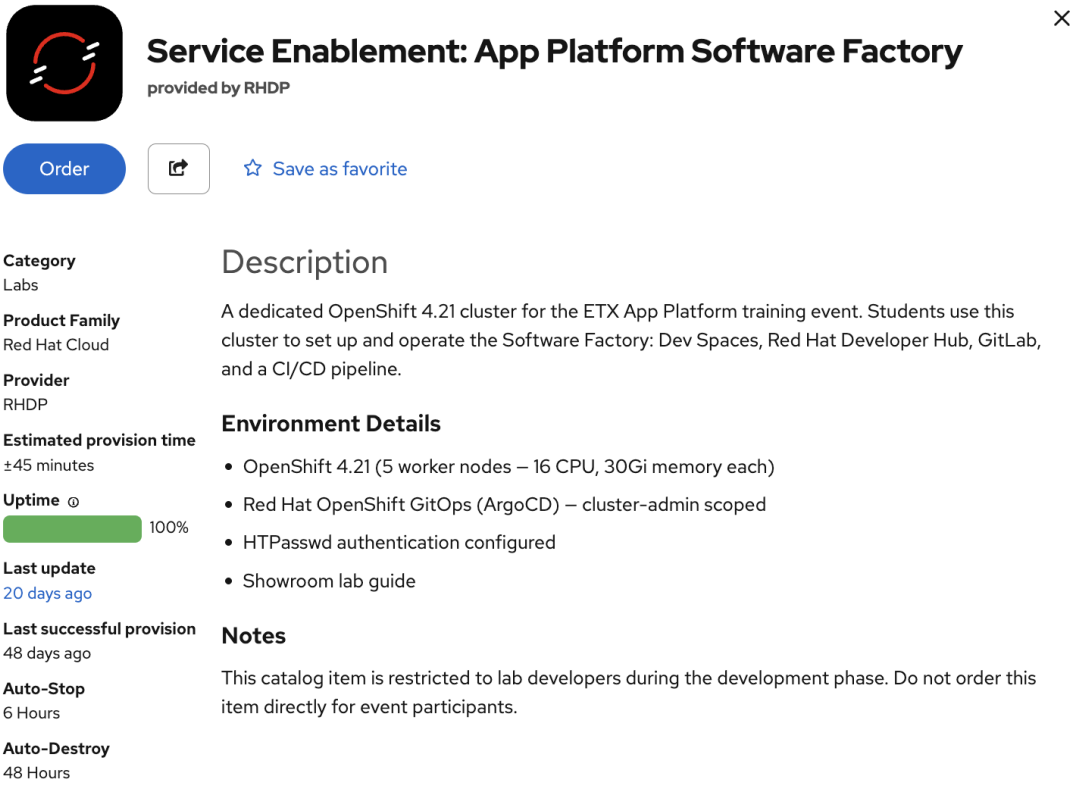
- RHDP for RedHat associates: https://catalog.demo.redhat.com/
- RHDP for RedHat partners: https://partner.demo.redhat.com/

Provision the Lab Environments

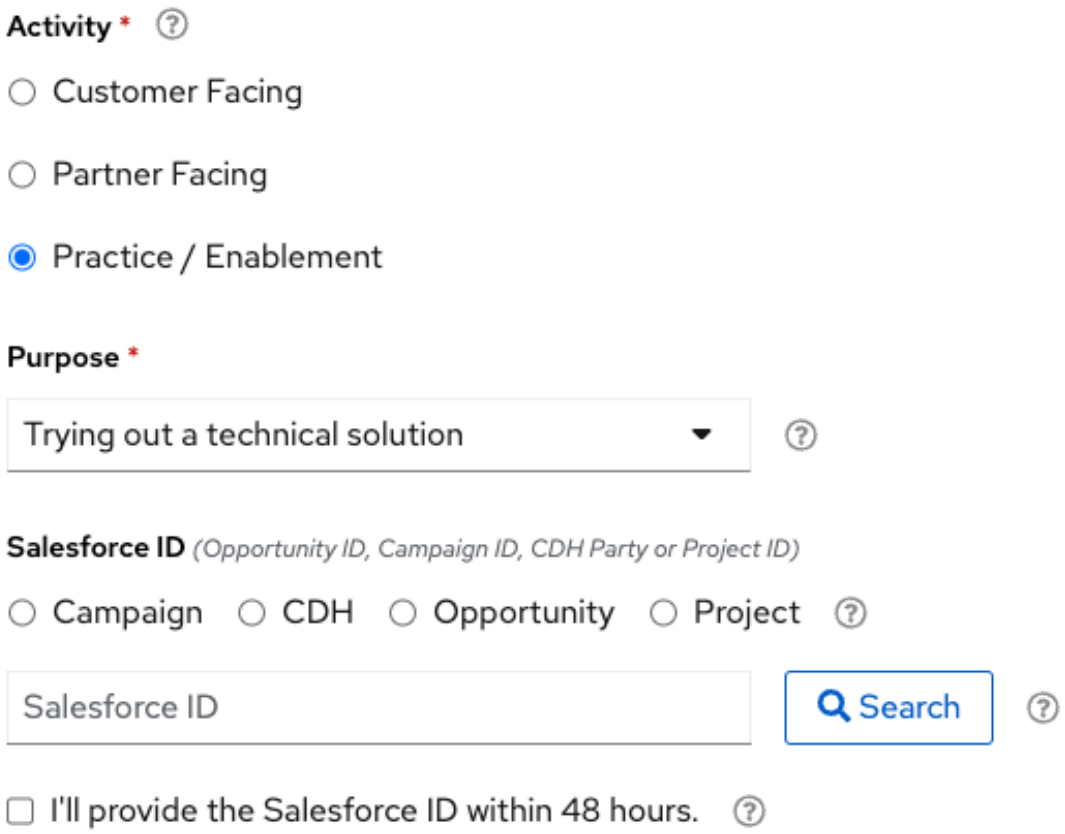
Provision the following RHDP catalog items:

Lab 1

1. **Log in** to the RHDP portal
2. **Search** for the catalog entry: **Service Enablement: App Platform Software Factory**.



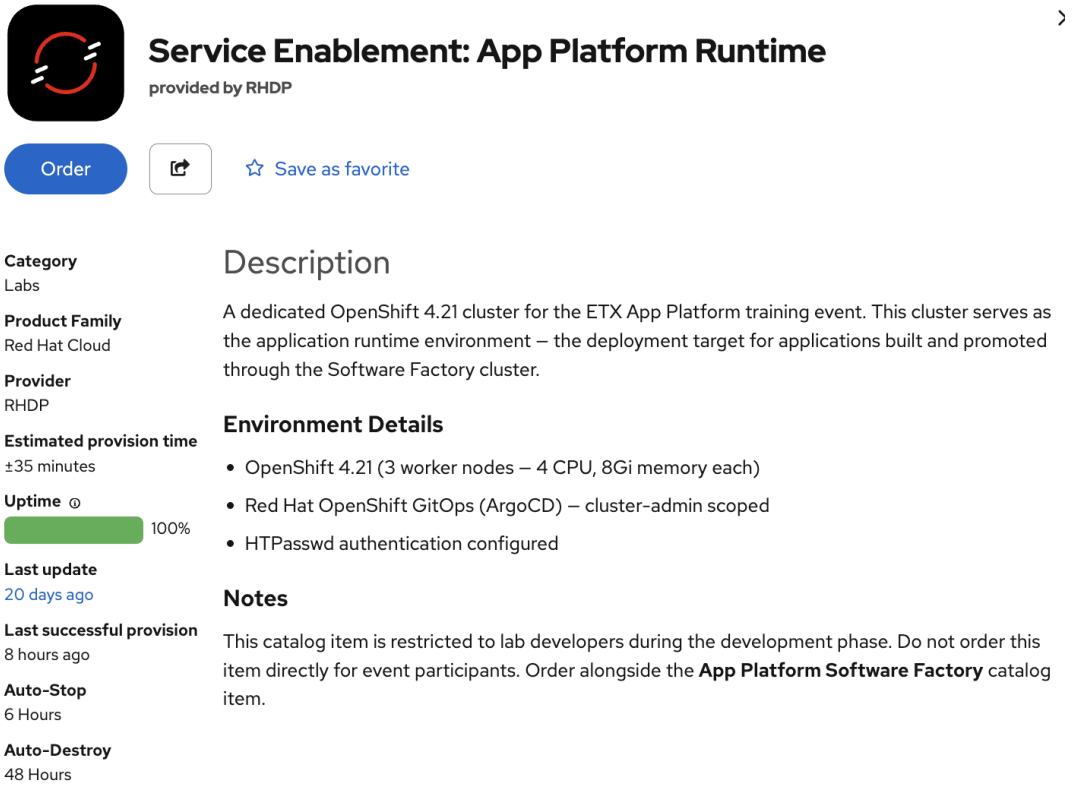
3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.



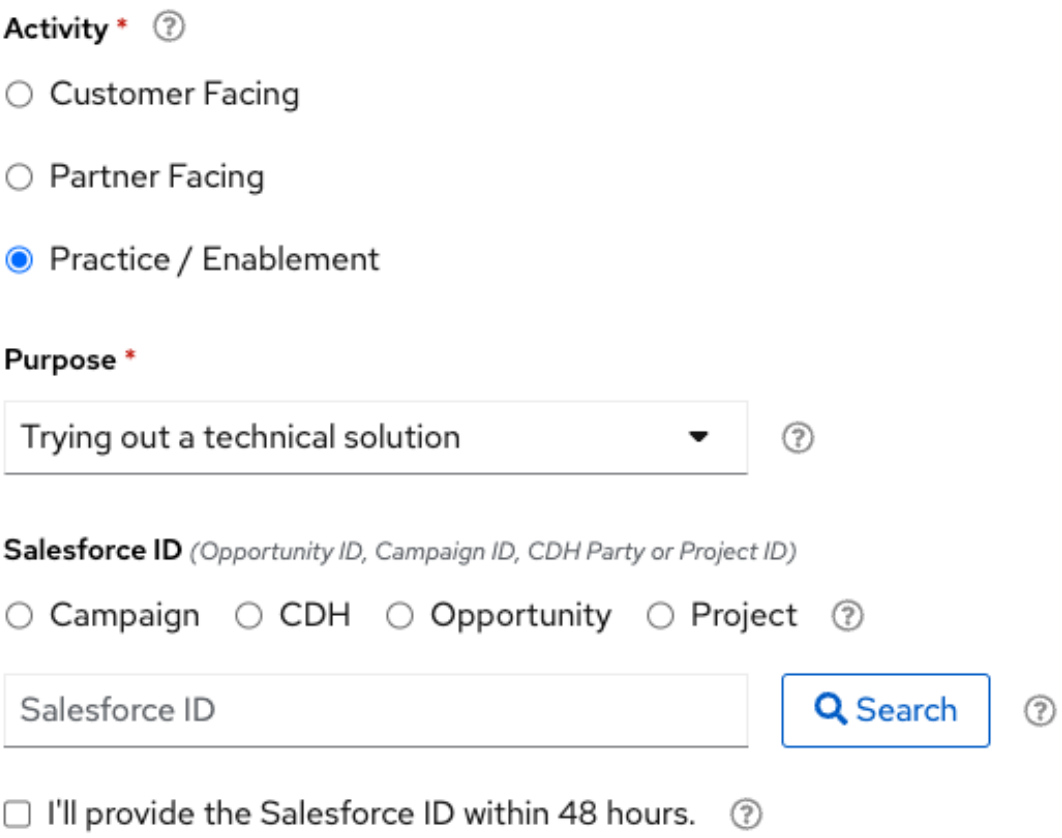
6. **Review the warning** at the bottom of the form and **check the box** labeled: **"I confirm that I understand the above warnings."**
7. **Click** the **Order** button to place your lab order.

Lab 2

1. **Log in** to the RHDP portal
2. **Search** for the catalog entry: **Service Enablement: App Platform Runtime**.



3. **Click** on the catalog entry in the search results.
4. On the catalog page, **click** the **Order** button.
5. **Fill out** the required details in the order form.



6. **Review the warning** at the bottom of the form and **check the box** labeled:

“I confirm that I understand the above warnings.”

7. **Click** the **Order** button to place your lab order.



Make sure you continue using the **same lab environment** provisioned for this module throughout the following modules:

- *Software Delivery with OpenShift Pipelines and GitOps*
- *DevEx with Red Hat Developer Hub*
- *Application Modernization with MTA*

Do **not delete, reset, or reprovision** the lab environment after completing this module. The following modules build progressively on the **applications, configurations, and platform resources** created in the previous modules.

Provisioning each environment typically takes about an hour. Once complete, the cluster URLs and credentials are available from the **My Services** page in RHDP.

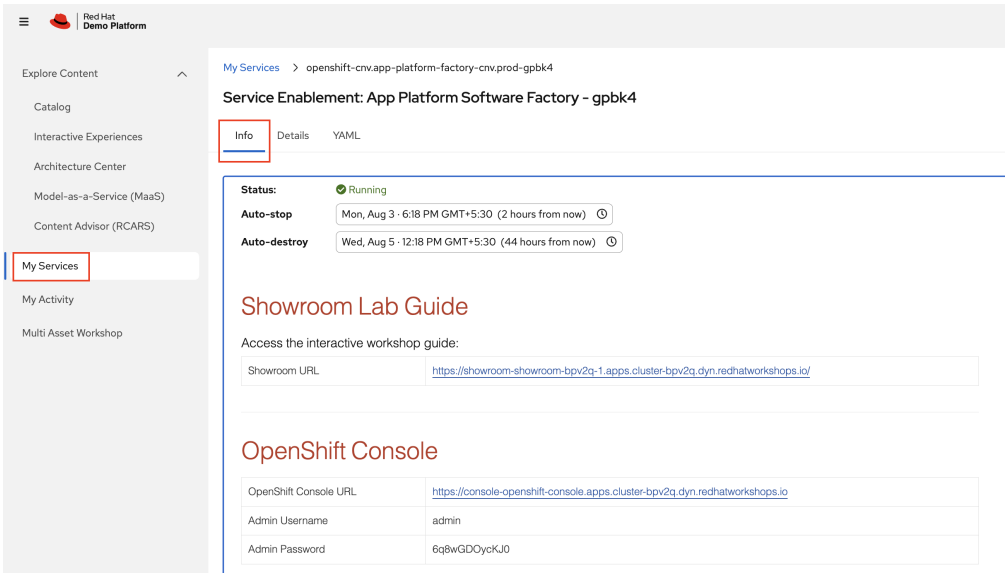


Figure 1. Sample **RHDP My Services** *Screenshot(Click the image to view it in full size in a new browser tab)*

OpenShift Clusters

Environment	Purpose	Used In
Service Enablement: App Platform Software Factory (Primary Cluster)	Development platform where you’ll build the internal developer platform, configure CI/CD, and deploy applications.	• *Software Factory Foundations with OpenShift Dev Spaces* • *Software Delivery with OpenShift Pipelines and GitOps* • *DevEx with Red Hat Developer Hub* • *Application Modernization with MTA*

Environment	Purpose	Used In
Service Enablement: App Platform Runtime (Secondary Cluster)	Runtime environment used to demonstrate GitOps-based promotion between environments.	• *Software Delivery with OpenShift Pipelines and GitOps*

The required namespaces have already been created and are ready for use.

Internal Lab Account Credentials

Use the following accounts throughout the learning path.

Username	Password	Purpose
`platform-admin`	`openshift`	Platform administrator
`platform-developer`	`openshift`	Application developer
`platform-user`	`openshift`	Standard user

Unless instructed otherwise, authenticate to GitLab, Quay, Vault, and other integrated services using **Keycloak Single Sign-On (SSO)**.

Platform Services

The following services are pre-configured and available throughout the labs.

Service	Purpose
Red Hat Build of Keycloak	Provides centralized authentication and Single Sign-On (SSO).
GitLab	Hosts the application source code and Git repositories.
Red Hat Quay	Stores container images built during the labs.
OpenShift GitOps	Continuously synchronizes applications and platform resources from Git.
HashiCorp Vault	Securely stores secrets used by applications and CI/CD pipelines.

All service URLs are available from the **Red Hat Demo Platform (RHDP)** **Lab Information** in **My Services** section page.

Software Factory Overview



What's in this section?

In the previous module, you learned how an **Application Platform** provides the capabilities required to build, deploy, secure, and operate modern applications.

In this section, you'll discover how organizations use those capabilities to create a **Software Factory**—a standardized software delivery process that enables developers to build and deliver applications quickly, securely, and consistently.

Introduction

The Challenge

Every development team wants to deliver software quickly.

Unfortunately, many organizations evolve over time, resulting in different teams using different tools, different processes, and different engineering practices.



Figure 2. The Developer's (Consumers) Struggle(Click the image to view it in full size in a new browser tab)

A new developer joining the organization often needs to answer questions such as:

- Which Git repository should I use?

- Which build pipeline does this team use?
- How do I get my code deployed quickly?
- Where is the documentation?
- Which security requirements must I follow?
- What tools and processes do I use to validate my code?
- How do I get rapid feedback about my application?
- How do I know what software libraries are safe to use?
- How do I request new features for the platform?
- What needs to be changed to modernize my application?

Instead of writing code, developers spend valuable time learning processes and integrating tools.



When every team builds its own software delivery process, developer onboarding becomes slower, operational complexity increases, and engineering practices become inconsistent.

What is a Software Factory?

A **Software Factory** is a standardized approach for building, testing, securing, deploying, and operating software.

Rather than every development team assembling its own toolchain, Platform Engineers provide a curated platform that offers reusable services, automation, and organizational guardrails.

Developers simply consume these capabilities through a consistent self-service experience.

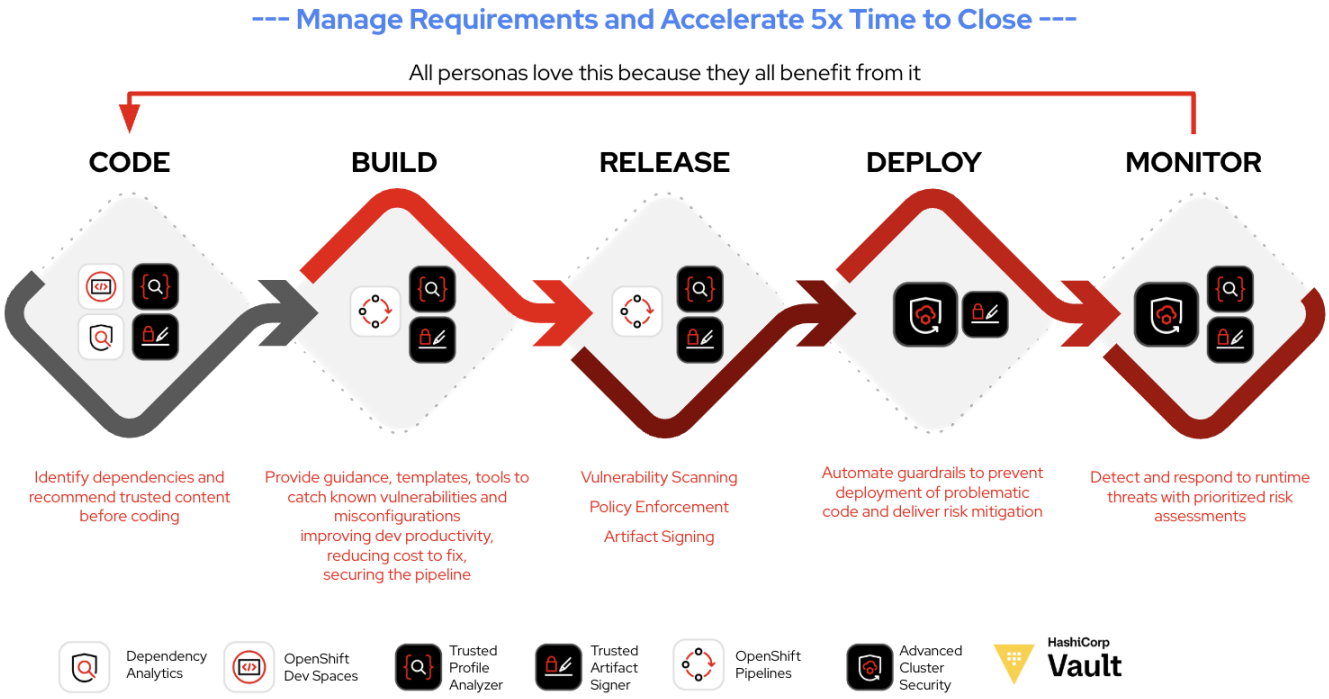


Figure 3. Trusted Software Factory(Click the image to view it in full size in a new browser tab)

Software Delivery Lifecycle

As application code moves from development to production, the Software Factory performs many tasks automatically.

Stage	Platform Activities
Code	Developers use standardized workspaces, templates, and source control.
Build	Applications are built, dependencies are validated, and security checks run automatically.
Release	Artifacts are signed, verified, and promoted according to organizational policies.
Deploy	Applications are deployed using automated pipelines and GitOps workflows.
Monitor	The platform continuously monitors application health, logs, and runtime behavior.

Business Value

By standardizing software delivery, organizations gain:

- Faster developer onboarding
- Consistent engineering practices
- Self-service development workflows
- Built-in security and compliance
- Automated software delivery
- Reduced operational complexity



A **Software Factory** is not a single product.

It is a standardized software delivery approach built on an Application Platform that combines automation, reusable services, and organizational best practices into a consistent developer experience.

Reflection

▼ *Think about your own organization (Click to expand)*



- Does every development team follow the same software delivery process?
- Which activities are already automated?
- Which steps still require manual effort?

- How could a standardized Software Factory improve developer productivity?

Key Takeaway

A **Software Factory** standardizes software delivery by providing reusable workflows, automation, and built-in engineering practices, allowing developers to focus on building applications instead of integrating tools.

A Day in the Software Factory

Now that you've learned what a Software Factory is, let's see how it helps a developer during everyday software development.

In this lesson, you'll follow Emma as she delivers her first application using her organization's Software Factory.



As you read through Emma's journey, think about how similar steps are performed in your own organization.

Meet Emma



Emma has just joined the engineering team.

She has been assigned her first feature and wants to start contributing immediately.

Instead of requesting infrastructure, configuring development environments, creating CI/CD pipelines, and learning deployment processes, Emma uses the organization's Software Factory.

The platform provides everything she needs through a standardized self-service experience.

Emma's Journey

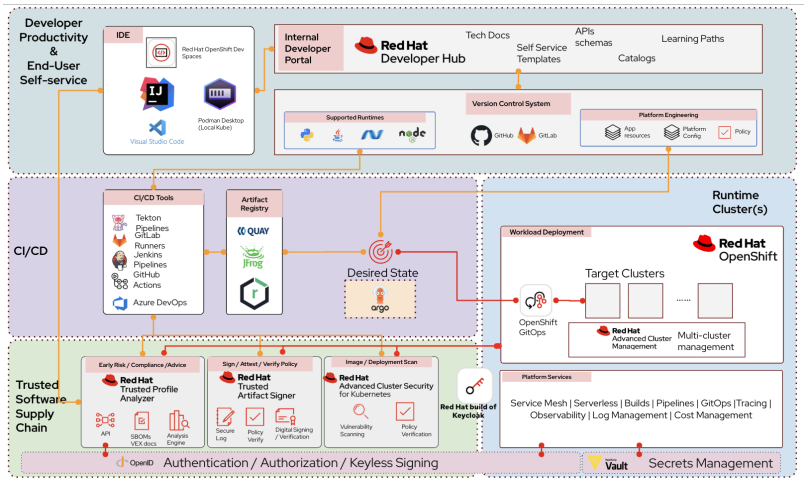


Figure 4. Application Platform Tools(Click the image to view it in full size in a new browser tab)

```
<div style="margin-bottom: 2rem;"></div>
```

▼ 🧑‍💻 *Emma's workflow (Click to expand)*

Emma's first day looks like this:

1. Sign in to the organization's **Developer Hub**.
2. Choose a **Golden Path** for the application she wants to build.
3. Launch a ready-to-use development workspace using **Dev Spaces**.
4. Write code and commit the changes to Git.
5. The platform automatically builds and validates the application.
6. Security and compliance checks run automatically.
7. GitOps deploys the application to OpenShift.
8. The platform continuously monitors the running application.

Behind Every Step

Emma interacts with only a few simple interfaces, but many platform capabilities work together behind the scenes.

Emma's Activity	Platform Capability
Find project templates	Developer Hub
Start coding	Dev Spaces
Store source code	GitHub or GitLab
Build and test applications	OpenShift Pipelines
Perform security validation	Trusted Software Supply Chain
Store trusted artifacts	Quay
Deploy applications	GitOps
Monitor running applications	Platform Services

Why This Matters

Without a Software Factory, Emma would need to:

- Learn multiple development processes.
- Configure development tools.
- Build deployment pipelines.
- Understand infrastructure.
- Integrate security tooling.

With a Software Factory, these capabilities are already provided by the platform.

Emma focuses on delivering business features while the platform handles the underlying automation, security, and operational tasks.



A Software Factory doesn't replace developers—it removes the repetitive platform work that slows them down, allowing developers to focus on writing application code.

Reflection

▼ *Think about your own organization (Click to expand)*



When a new developer joins your team:

- How long does onboarding take?
- Is there a standardized workflow?
- Which activities are manual?
- Which activities could be automated through a Software Factory?

Version Control

What's in this section?

In this section, you'll learn why **Git version control** is a foundational capability of a modern Application Platform and how it enables collaborative software development, CI/CD pipelines, and GitOps-based deployments.

> If you're already comfortable using Git and common branching strategies, feel free to skim this lesson and continue to Lab 1: Initial Setup. The concepts introduced here provide useful context for the Software Factory you'll build throughout this course.

Collaborative Development

Connecting the Dots

Once developers write code, they need a reliable way to share changes, review code, maintain version history, and trigger automated software delivery workflows.

This is where **Git** becomes an essential part of the Software Factory.

[1*S4VJPf8Mrg3Gn5cR7JK62w] |
*https://miro.medium.com/v2/resize:fit:720/format:webp/1*S4VJPf8Mrg3Gn5cR7JK62w.png*

Figure 5. Git Workflow for Software Delivery(Click the image to view it in full size in a new browser tab)

> Throughout this learning path, Git serves as the source of truth for your application code. Every code change committed to Git can automatically trigger builds, tests, deployments, and GitOps workflows.

Why Version Control Matters

Modern applications are rarely developed by a single developer.

Multiple developers often work on the same application simultaneously, making it essential to track changes, collaborate safely, and maintain a complete history of the application.

Version control systems such as **Git** make this possible by recording every change made to the

source code.

Instead of storing only the latest version of an application, Git maintains a complete history that allows teams to:

- Collaborate without overwriting each other's work
- Track who made changes and when
- Review code before it is merged
- Restore previous versions when necessary
- Develop features independently
- Trigger automated software delivery pipelines



Git is much more than a code repository. It provides the collaboration and version history that modern Application Platforms rely on to automate software delivery.

Understanding Branches

A **branch** represents an isolated line of development.

Rather than making changes directly to the main codebase, developers create a separate branch where they can safely implement new features, fix defects, or experiment with ideas.

Once the work has been reviewed and validated, the branch is merged back into the main branch.

[git model@2x] | *https://nvie.com/img/git-model@2x.png*

Figure 6. Git branching strategy(Click the image to view it in full size in a new browser tab)



Throughout the hands-on labs, you'll create feature branches, commit changes, and merge them into the main branch to trigger automated Software Factory workflows.

Common Branching Strategies

Different organizations adopt different branching models depending on their release process, team size, and deployment frequency.

Branching Strategy	Characteristics	Common Use Cases
Trunk-Based Development	Developers create short-lived branches and merge changes into the main branch frequently.	Continuous Integration and Continuous Delivery
GitHub Flow	Feature branches combined with Pull or Merge Requests before merging into the main branch.	Cloud-native applications and open source projects

Branching Strategy	Characteristics	Common Use Cases
GitFlow	Multiple long-lived branches for development, releases, and hotfixes.	Traditional enterprise applications with scheduled releases

Branching Strategy Used in This Course

Throughout this learning path, you'll use a simplified Trunk-Based Development workflow.

[trunk1b] | *https://trunkbaseddevelopment.com/trunk1b.png*

Figure 7. Trunk-Based Development(Click the image to view it in full size in a new browser tab)

This approach keeps the focus on learning Application Platform concepts rather than Git administration.

The workflow you'll follow is:

1. Create a feature branch
2. Make code changes
3. Commit your changes
4. Push the branch to Git
5. Merge the branch into the main branch
6. Automatically trigger the Software Factory



You'll practice this workflow during the upcoming hands-on lab, where each Git commit becomes the starting point for automated build and deployment pipelines.

Reflection

▼ *Think about your own organization (Click to expand)*



- Which version control system does your organization use?
- Which branching strategy is followed by your team?
- Are code changes automatically validated through CI/CD?
- How frequently are changes merged into the main branch?

```
<div class="key-card">
```

Key Takeaway

In a modern Software Factory, **Git is often the event that starts the entire software delivery process**. A single commit can automatically trigger builds, testing, security validation, deployments, and GitOps synchronization.

Lab1: Initial Setup

In this lab, you'll access the Software Factory environment and prepare the application that will be used throughout the remaining exercises.

By the end of this lab, you will have:

- Authenticated to GitLab using enterprise Single Sign-On
- Cloned the workshop application repository
- Verified access to the OpenShift cluster

Sign in to GitLab

The training uses **GitLab** as the central source code repository. Authentication is handled through **Keycloak** using **Single Sign-On (SSO)**.

1. Navigate to the **GitLab URL** provided on your RHDP lab information page (for example, `https://gitlab.apps.cluster-GUID.dyn.redhatworkshops.io`).
2. On the GitLab sign-in page, click **[ETX Lab SSO]**.

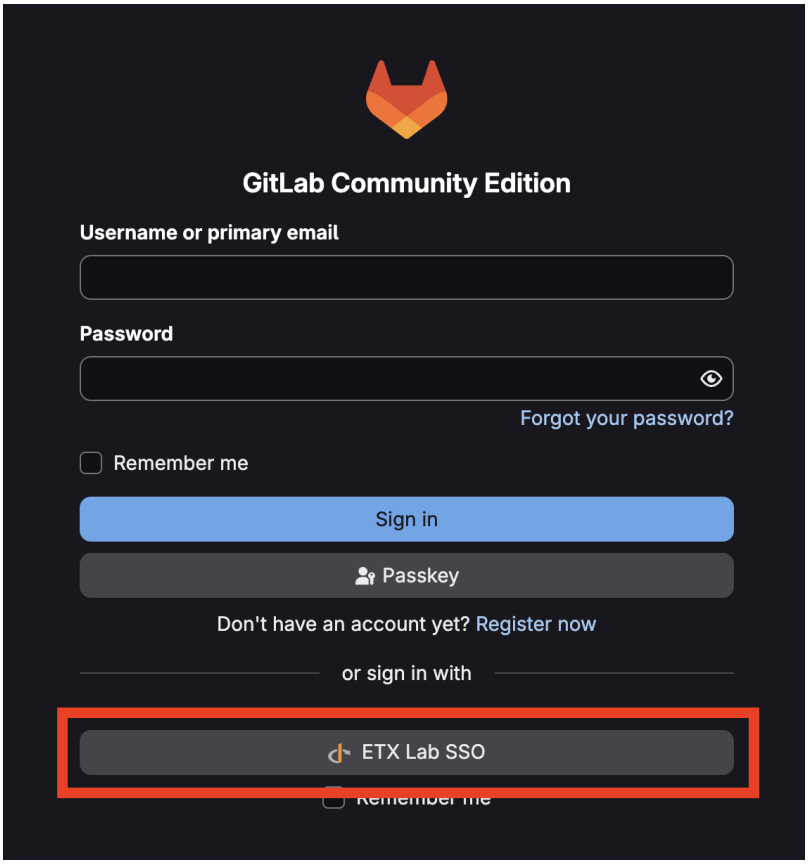


Figure 8. Signin into GitLab(Click the image to view it in full size in a new browser tab)

3. Sign in using any one of the provided credentials in the Lab Credentials section (for example, `username: platform-developer` / `password: openshift`).
4. After successful authentication, you'll be redirected back to GitLab.

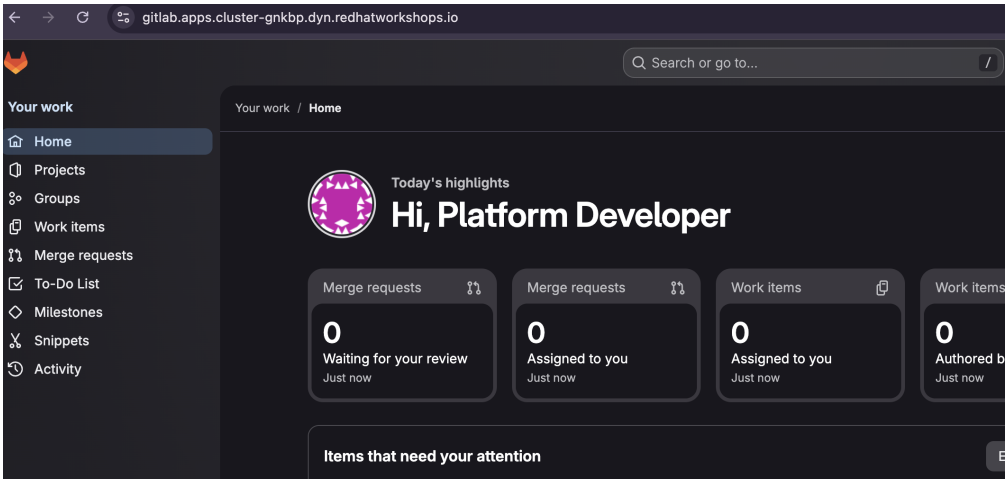


Figure 9. Sample GitLab Home Page(Click the image to view it in full size in a new browser tab)



The first time you sign in, GitLab automatically creates a user profile linked to your Keycloak identity.

Clone the Workshop Repository

Throughout the learning path, you'll work with the **ETX App Base** application.

Before cloning the repository, sign in to the OpenShift web console.

1. Open the **OpenShift Console URL** listed on the **RHDP Lab Environment** page and sign in using any one of the credentials listed in the **Info** page.
2. Click the >_ (**Web Terminal**) icon in the upper-right corner to launch a terminal session.

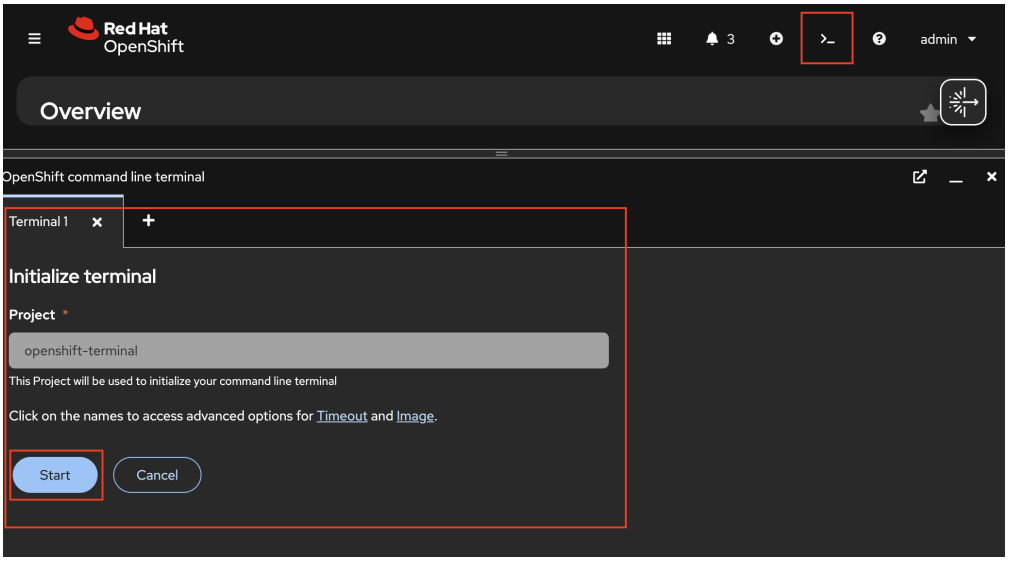


Figure 10. Web Terminal in Openshift console(Click the image to view it in full size in a new browser tab)

Retrieve the **GitLab** route from your OpenShift cluster and store it in a variable for use in next steps.

```
GITLAB_URL="https://$(oc get route \
  -n gitlab \
  -l app=webservice \
```

```
  -o jsonpath='{.items[0].spec.host}')"

echo "${GITLAB_URL}"
```

Clone the application repository.

```
mkdir -p ~/workshop
cd ~/workshop

git clone ${GITLAB_URL}/software-factory/etx_app_base_app.git
cd etx_app_base_app
```



The Git repository is named `etx_app_base_app`. Kubernetes resources created later in the training use the naming convention `etx-app-base-app-*`.



This repository intentionally does **not** contain a `devfile.yaml`. In the next lab, you'll create one yourself and use it to launch a Dev Spaces workspace.

Verify OpenShift Access

Confirm that you're connected to the openshift cluster and have access to the development project.

```
oc whoami --show-server

oc project etx-app-dev
```

Deploy Supporting Services



What's in this section?

In this section, you prepare the application platform services required by installing the required operators, deploy the supporting infrastructure, and verify that each service is running and ready for use in the remaining labs.

Lab 2: Red Hat OpenShift Pipelines

In this lab, you'll install the Red Hat OpenShift Pipelines Operator from OperatorHub and verify that the Tekton platform components have been successfully deployed.

By the end of this lab, you will have:

- Installed the Red Hat OpenShift Pipelines Operator
- Verified that the operator installed successfully
- Confirmed that the Tekton controllers are running

Install the Red Hat OpenShift Pipelines Operator

OpenShift Pipelines is delivered as an Operator that installs and manages the Tekton components required for building CI/CD pipelines.

1. Open the OpenShift web console.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

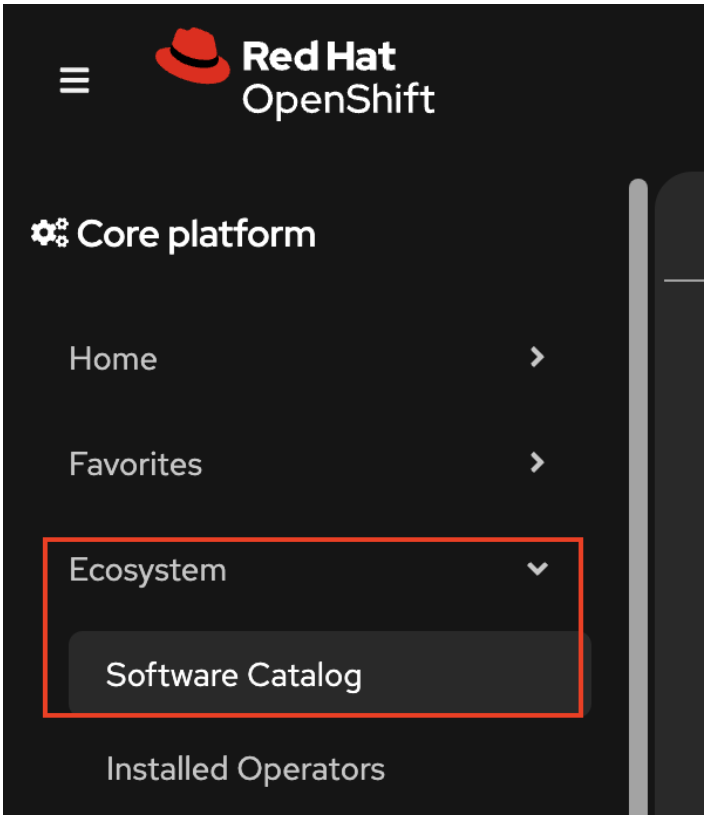


Figure 11. Ecosystem → Software Catalog(Click the image to view it in full size in a new browser tab)

4. Search for **Red Hat OpenShift Pipelines**.

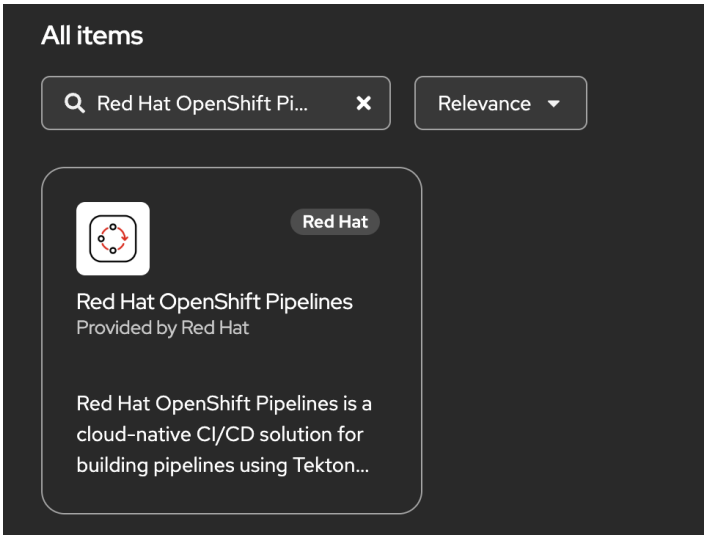


Figure 12. Red Hat OpenShift Pipelines Operator(Click the image to view it in full size in a new browser tab)



It may take a few moments for OperatorHub to finish loading all available operators.

5. Select the operator and click **[Install]**.

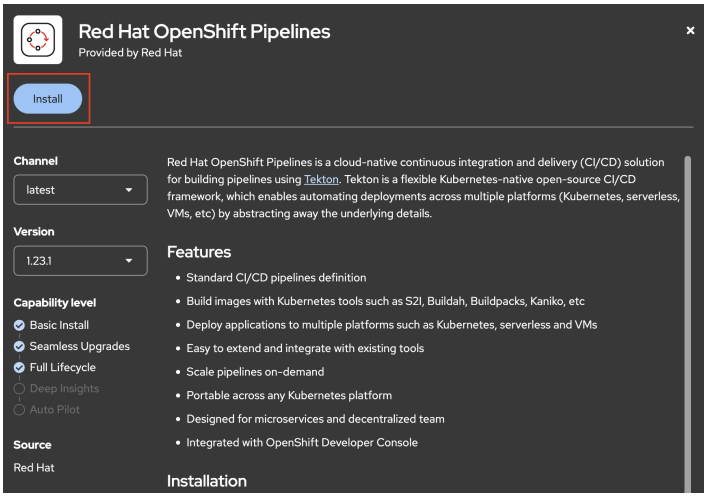


Figure 13. Red Hat OpenShift Pipelines Install(Click the image to view it in full size in a new browser tab)

6. Accept the default installation settings:
    - **Update channel:** `latest`
    - **Installation mode:** `All namespaces on the cluster`
    - **Installed namespace:** `openshift-operators`
7. Click **[Install]**.

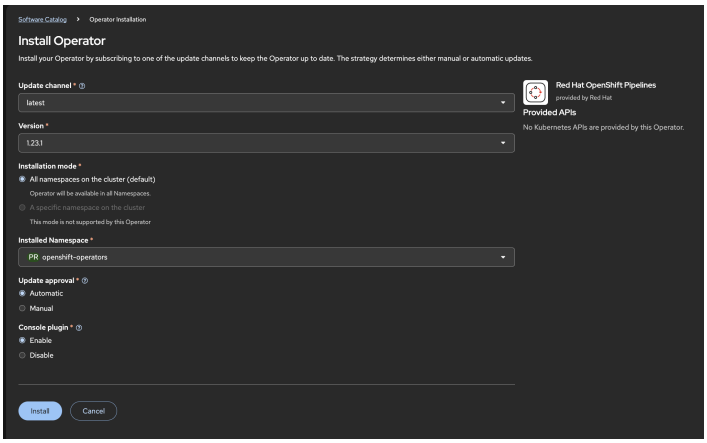


Figure 14. Red Hat OpenShift Pipelines Install(Click the image to view it in full size in a new browser tab)

8. Wait until the operator status changes to **Succeeded**.

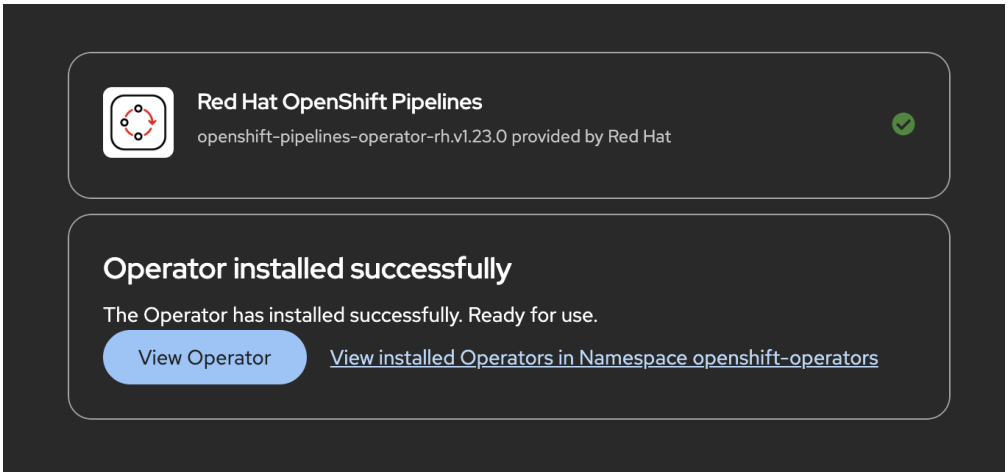


Figure 15. Red Hat OpenShift Pipelines Operator(Click the image to view it in full size in a new browser tab)



Installing the operator deploys the Tekton controllers and supporting components that provide the pipeline execution engine used throughout the remaining labs.

Verify the Operator Installation

Confirm that the OpenShift Pipelines Operator has been installed successfully.

```
oc get csv -n openshift-operators | grep pipelines
```

Verify that the command returns the OpenShift Pipelines ClusterServiceVersion (CSV) with a status of `Succeeded`.

Verify the Tekton Components

Next, verify that the Tekton platform components are running.

```
oc get pods -n openshift-pipelines
```

You should see the Tekton controller pods in the `Running` state, including:

- `tekton-pipelines-controller`
- `tekton-triggers-controller`
- `tekton-events-controller`
- `tekton-webhook`



Installing the operator automatically deploys the Tekton controllers and supporting components into the `openshift-pipelines` namespace. These components will execute the CI/CD pipelines you create later in this training.

Lab 3: Red Hat AMQ Streams

In this lab, you'll install the Red Hat AMQ Streams Operator, deploy a single-node Apache Kafka cluster, and verify that it is ready for use by the training application.

Red Hat AMQ Streams provides Apache Kafka as a managed service on OpenShift. Kafka enables applications to exchange messages asynchronously, making it easier to build scalable, event-driven applications.

By the end of this lab, you will have:

- Installed the Red Hat AMQ Streams Operator
- Verified that the operator installed successfully
- Deployed a single-node Kafka cluster
- Confirmed that the Kafka cluster is ready to accept client connections

Install the Red Hat AMQ Streams Operator

Install the Red Hat AMQ Streams Operator from OperatorHub. This operator installs and manages the Kafka components required by the training application.

1. Open the OpenShift web console.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

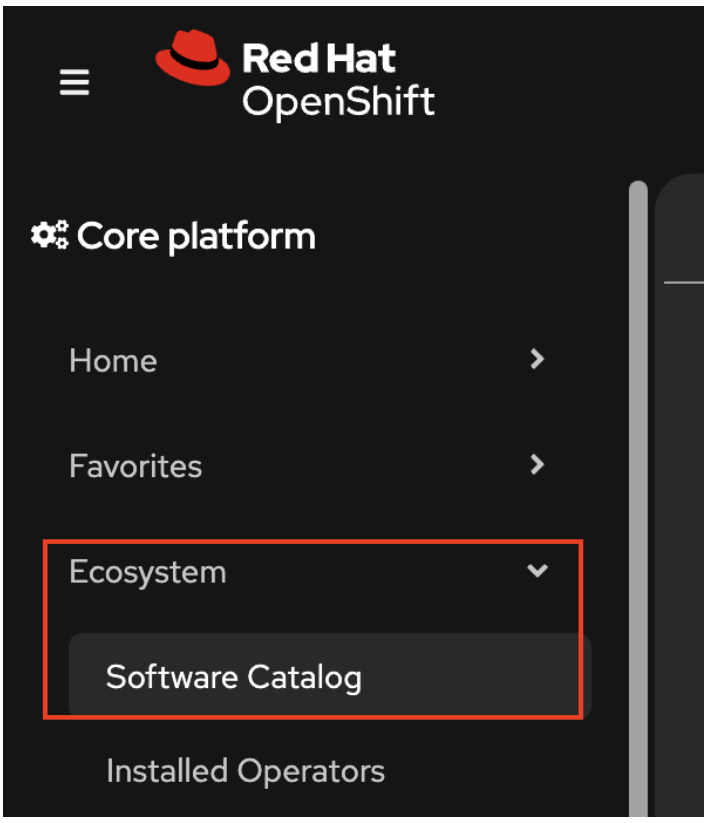


Figure 16. Ecosystem → Software Catalog(Click the image to view it in full size in a new browser tab)

4. Search for **Streams for Apache Kafka**.

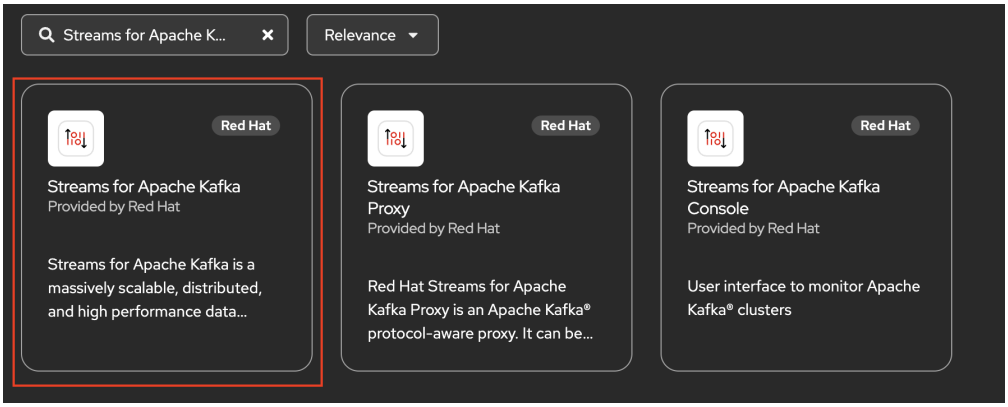


Figure 17. Streams for Apache Kafka(Click the image to view it in full size in a new browser tab)



Install **Streams for Apache Kafka** only. Do **not** install **Streams for Apache Kafka Proxy**.

5. Select the operator and click **[Install]**.
6. Accept the default installation settings:
   - **Update channel:** `stable`
   - **Installation mode:** `All namespaces`
   - **Installed namespace:** `openshift-operators`

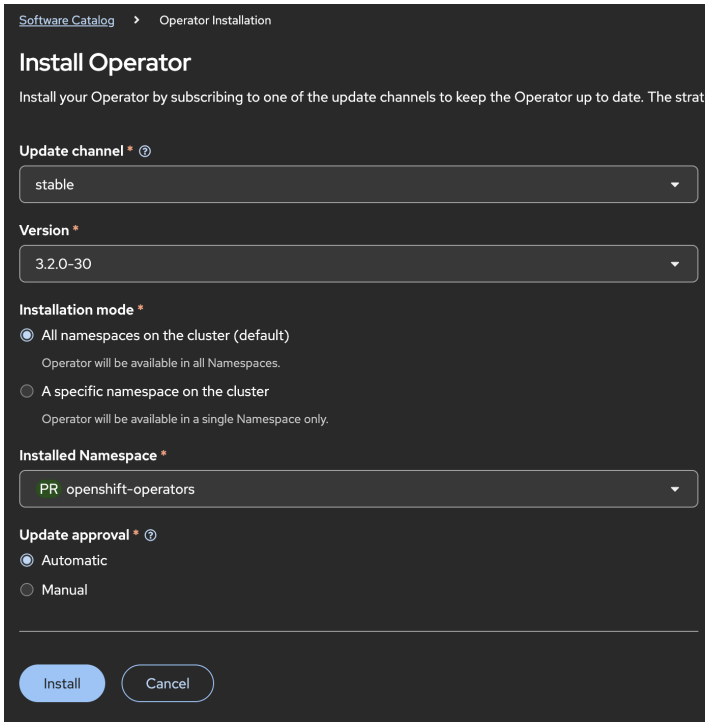


Figure 18. Install Streams for Apache Kafka(Click the image to view it in full size in a new browser tab)

7. Click **[Install]**.
8. Wait until the operator status changes to **Succeeded**.

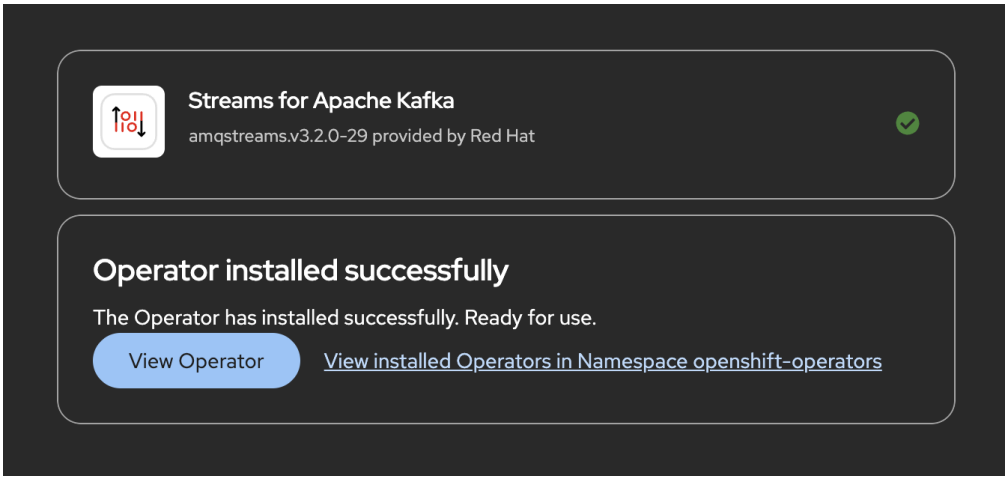


Figure 19. Red Hat AMQ Streams Operator(Click the image to view it in full size in a new browser tab)

Installing the operator deploys the components required to create and manage Apache Kafka clusters on OpenShift.

Verify the Operator Installation

Confirm that the Red Hat AMQ Streams Operator has been installed successfully.

```
oc get csv -n openshift-operators | grep amqstreams
```

Verify that the command returns the AMQ Streams ClusterServiceVersion (CSV) with a status of `Succeeded`.

Deploy a Kafka Cluster

With the operator installed, you can now deploy a Kafka cluster in the `etx-app-dev` project.

Starting with AMQ Streams 3.x, Kafka clusters use **KafkaNodePools** to manage broker nodes. A KafkaNodePool defines the Kafka nodes, their roles, and storage configuration.

In this training, you'll deploy a single-node Kafka cluster running in **KRaft mode**, where the same node acts as both the controller and broker. This removes the need for ZooKeeper and simplifies the deployment for development environments.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: kafka.strimzi.io/v1
kind: KafkaNodePool
metadata:
  name: kafka
  namespace: etx-app-dev
  labels:
    strimzi.io/cluster: parasol-kafka
spec:
```

```
  replicas: 1
  roles:
    - controller
    - broker
  storage:
    type: ephemeral
---
apiVersion: kafka.strimzi.io/v1
kind: Kafka
metadata:
  name: parasol-kafka
  namespace: etx-app-dev
  annotations:
    strimzi.io/node-pools: enabled
    strimzi.io/kraft: enabled
spec:
  kafka:
    version: 4.2.0
    metadataVersion: 4.2-IV0
    listeners:
      - name: plain
        port: 9092
        type: internal
        tls: false
      - name: tls
        port: 9093
        type: internal
        tls: true
    config:
      offsets.topic.replication.factor: 1
      transaction.state.log.replication.factor: 1
      transaction.state.log.min.isr: 1
      default.replication.factor: 1
      min.insync.replicas: 1
  entityOperator:
    topicOperator: {}
    userOperator: {}
EOF
```

Verify the Kafka Cluster

Wait for the Kafka cluster to become ready.

```
oc wait kafka/parasol-kafka \
  --for=condition=Ready \
  --timeout=300s \
  -n etx-app-dev
```

After the cluster reports a `Ready` status, verify that the Kafka pods are running.

```
oc get pods \
  -l strimzi.io/cluster=parasol-kafka \
  -n etx-app-dev
```

The output should include:

- `parasol-kafka-kafka-0`
- `parasol-kafka-entity-operator-*`



Ensure all Kafka pods are in the `Running` state before proceeding.

Verify the Bootstrap Service

Applications connect to Kafka through a bootstrap service that provides the connection endpoint for Kafka clients.

Retrieve the bootstrap service endpoint.

```
oc get service parasol-kafka-kafka-bootstrap \
  -n etx-app-dev \
  -o jsonpath='{.metadata.name}:{.spec.ports[0].port}{"\n"}'
```

The command should return:

```
parasol-kafka-kafka-bootstrap:9092
```



You'll use this bootstrap endpoint later in the training when configuring the application to communicate with Kafka.

Lab 4: PostgreSQL Database

In this lab, you'll deploy a PostgreSQL database that provides persistent storage for the training application.

The application uses PostgreSQL to store and retrieve application data. You'll deploy the database using the Red Hat certified PostgreSQL container image and verify that it is ready to accept client connections.

By the end of this lab, you will have:

- Deployed a PostgreSQL database on OpenShift
- Verified that the database deployment is running
- Confirmed that applications can connect to the database

Deploy the PostgreSQL Database

Deploy PostgreSQL along with the Kubernetes resources required by the application, including a Secret for the database credentials and a Service for internal connectivity.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: v1
kind: Secret
metadata:
  name: parasol-db-secret
  namespace: etx-app-dev
type: Opaque
stringData:
  database-name: parasol
  database-user: parasol
  database-password: parasol
---
apiVersion: v1
kind: Service
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  ports:
    - name: postgresql
      port: 5432
      targetPort: 5432
  selector:
    app: parasol-db
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: parasol-db
  namespace: etx-app-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: parasol-db
  template:
    metadata:
      labels:
        app: parasol-db
    spec:
      containers:
        - name: postgresql
          image: registry.redhat.io/rhel9/postgresql-16:latest
```

```
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRESQL_USER
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-user
            - name: POSTGRESQL_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-password
            - name: POSTGRESQL_DATABASE
              valueFrom:
                secretKeyRef:
                  name: parasol-db-secret
                  key: database-name
          volumeMounts:
            - name: postgresql-data
              mountPath: /var/lib/pgsql/data
          livenessProbe:
            exec:
              command:
                - /usr/libexec/check-container
                - --live
            initialDelaySeconds: 120
            timeoutSeconds: 10
          readinessProbe:
            exec:
              command:
                - /usr/libexec/check-container
            initialDelaySeconds: 5
            timeoutSeconds: 1
          resources:
            limits:
              memory: 512Mi
              cpu: 500m
            requests:
              memory: 256Mi
              cpu: 100m
      volumes:
        - name: postgresql-data
          emptyDir: {}
EOF
```



This deployment uses `emptyDir` for storage, which means all database data is lost if the pod is restarted. In production environments, use a `PersistentVolumeClaim` backed by persistent storage.

Verify the PostgreSQL Deployment

Wait for the PostgreSQL deployment to complete successfully.

```
oc rollout status deployment/parasol-db -n etx-app-dev
```

After the rollout completes, verify that the PostgreSQL pod is running.

```
oc get pods \
  -l app=parasol-db \
  -n etx-app-dev
```

Verify that the PostgreSQL pod is in the `Running` state before continuing.

Verify Database Connectivity

Confirm that applications can connect to the PostgreSQL database.

```
oc run postgresql-test --rm -it --restart=Never \
  --image=registry.redhat.io/rhel9/postgresql-16:latest \
  --env PGPASSWORD=$(oc get secret -n etx-app-dev parasol-db-secret -o
jsonpath='{.data.database-password}' | base64 -d) \
  -n etx-app-dev -- \
  psql -h parasol-db \
  -U $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
user}' | base64 -d) \
  -d $(oc get secret -n etx-app-dev parasol-db-secret -o jsonpath='{.data.database-
name}' | base64 -d) \
  -c '\conninfo'
```

If the connection is successful, you'll see information similar to the following:

```
You are connected to database "parasol" as user "parasol" on host "parasol-db" at port
"5432".
```



The database credentials are stored in the `parasol-db-secret` Secret. Later in this training, the application will use these same credentials to establish its database connection.

Lab 5: Red Hat Build of OpenTelemetry

In this lab, you'll install the Red Hat Build of OpenTelemetry Operator, deploy an OpenTelemetry Collector, and verify that it is ready to receive telemetry data from applications.

The Red Hat Build of OpenTelemetry provides the observability foundation for cloud-native

applications. It enables applications to collect, process, and export telemetry data such as distributed traces, metrics, and logs.

By the end of this lab, you will have:

- Installed the Red Hat Build of OpenTelemetry Operator
- Verified that the operator installed successfully
- Deployed an OpenTelemetry Collector
- Confirmed that the collector is ready to receive telemetry data

Install the Red Hat Build of OpenTelemetry Operator

Install the Red Hat Build of OpenTelemetry Operator from OperatorHub. This operator manages the lifecycle of OpenTelemetry Collectors and enables automatic instrumentation for supported applications.

1. Open the OpenShift web console.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

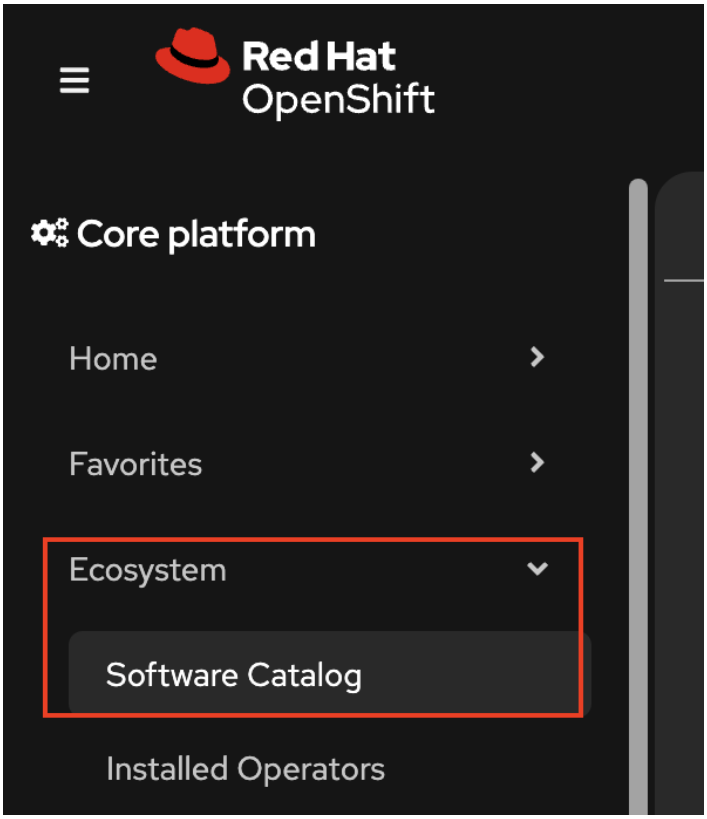


Figure 20. Ecosystem → Software Catalog(Click the image to view it in full size in a new browser tab)

4. Search for **Red Hat build of OpenTelemetry**.

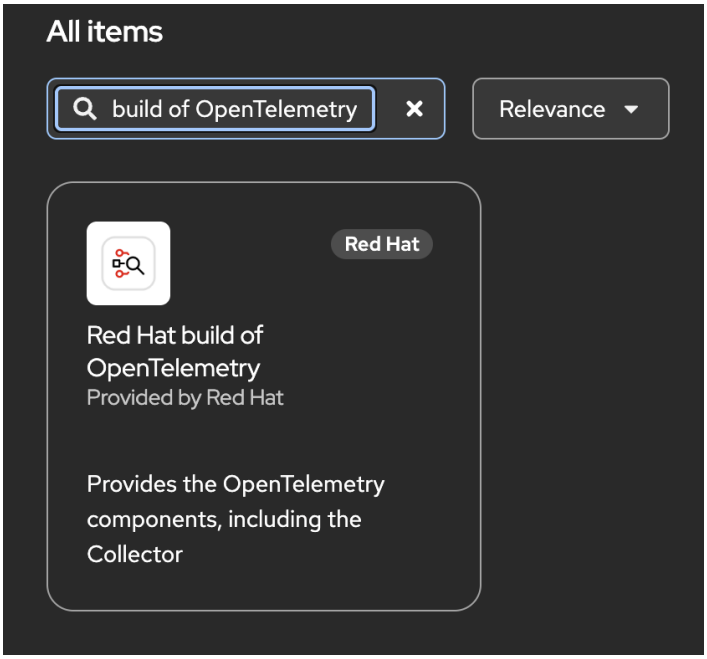


Figure 21. Red Hat build of OpenTelemetry Operator(Click the image to view it in full size in a new browser tab)

5. Select the operator and click **[Install]**.
6. Accept the default installation settings:
    - **Update channel:** `stable`
    - **Installation mode:** `All namespaces`
    - **Installed namespace:** `openshift-opentelemetry-operator`
7. Click **[Install]**.

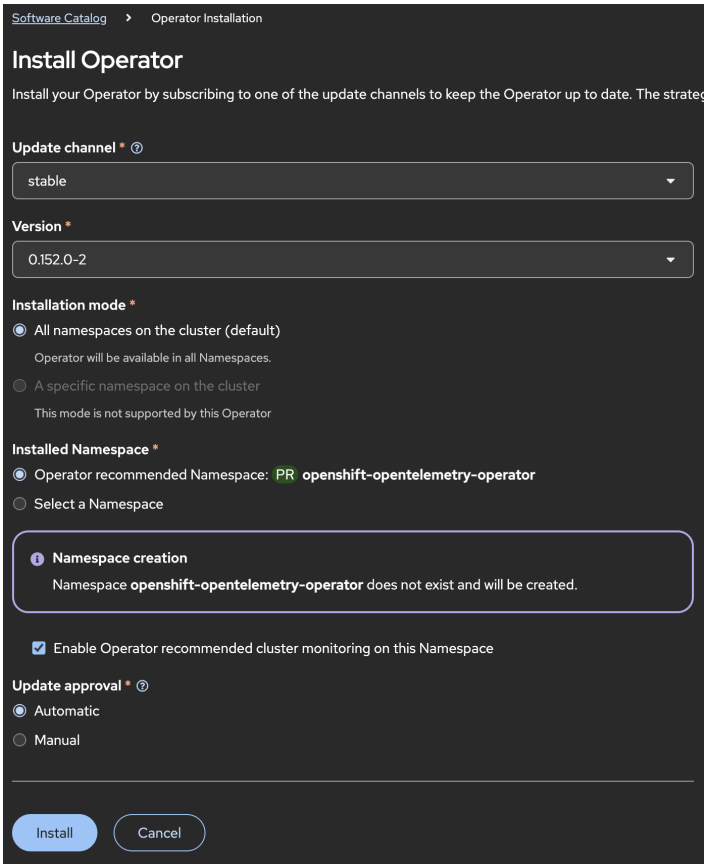


Figure 22. Install Red Hat build of OpenTelemetry Operator(Click the image to view it in full size in a new browser tab)

8. Wait until the operator status changes to **Succeeded**.

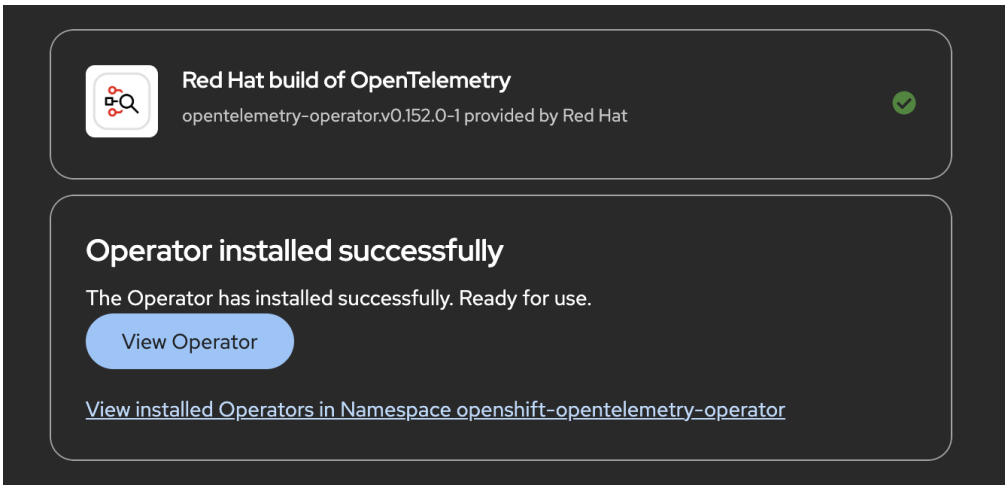


Figure 23. Red Hat Build of OpenTelemetry Operator(Click the image to view it in full size in a new browser tab)



Installing the operator enables you to deploy and manage OpenTelemetry Collectors and application instrumentation on OpenShift.

Verify the Operator Installation

Confirm that the Red Hat Build of OpenTelemetry Operator has been installed successfully.

```
oc get csv -n openshift-opentelemetry-operator
```

Verify that the command returns the OpenTelemetry Operator ClusterServiceVersion (CSV) with a status of `Succeeded`.

Deploy an OpenTelemetry Collector

An OpenTelemetry Collector receives telemetry data from instrumented applications, processes it, and exports it to one or more observability backends.

For this training, you'll deploy a collector that exports traces to its own logs. This provides a simple way to verify telemetry without requiring an external tracing platform.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1beta1
kind: OpenTelemetryCollector
metadata:
  name: otel-collector
  namespace: etx-app-dev
spec:
  mode: deployment
  config:
    receivers:
      otlp:
        protocols:
          grpc:
            endpoint: 0.0.0.0:4317
          http:
            endpoint: 0.0.0.0:4318
    processors:
      batch: {}
      probabilistic_sampler:
        sampling_percentage: 100.0
    exporters:
      debug:
        verbosity: detailed
    service:
      pipelines:
        traces:
          receivers: [otlp]
          processors: [probabilistic_sampler, batch]
          exporters: [debug]
EOF
```



The `debug` exporter writes trace information to the collector logs, making it suitable for development and demonstration environments.



In production, replace the `debug` exporter with a supported tracing backend such as Red Hat Build of Tempo, Jaeger, or another OpenTelemetry-compatible platform.

Verify the OpenTelemetry Collector

Wait for the OpenTelemetry Collector deployment to become available.

```
oc rollout status deployment/otel-collector-collector \
  -n etx-app-dev
```

After the deployment completes, verify that the collector pod is running.

```
oc get pods \
  -l app.kubernetes.io/name=otel-collector-collector \
  -n etx-app-dev
```

Verify that the collector pod is in the `Running` state.



Ensure the OpenTelemetry Collector is running before proceeding. Later in this training, the application will send telemetry data to this collector for observability.

Lab 6: Automatic Instrumentation

In this lab, you'll configure automatic instrumentation for Java applications using the Red Hat Build of OpenTelemetry Operator.

Automatic instrumentation enables applications to generate distributed traces without requiring any changes to the application source code. Instead, the OpenTelemetry Operator automatically injects a Java agent into the application when it is deployed.

By the end of this lab, you will have:

- Created an OpenTelemetry `Instrumentation` resource
- Configured automatic instrumentation for Java applications
- Verified that telemetry data can be exported to the OpenTelemetry Collector

Configure Automatic Instrumentation

The OpenTelemetry Operator uses an `Instrumentation` custom resource to define how applications should collect and export telemetry data.

In this training, you'll create an `Instrumentation` resource that configures Java applications to send traces to the OpenTelemetry Collector deployed in the previous lab.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: opentelemetry.io/v1alpha1
kind: Instrumentation
metadata:
  name: java-instrumentation
  namespace: etx-app-dev
  annotations:
    instrumentation.opentelemetry.io/inject-java: "true"
spec:
  exporter:
    endpoint: http://otel-collector-collector:4317
  propagators:
    - tracecontext
    - baggage
  sampler:
    type: always_on
  java:
    image: ghcr.io/open-telemetry/opentelemetry-operator/autoinstrumentation-
java:latest
EOF
```

The annotation `instrumentation.opentelemetry.io/inject-java: "true"` enables automatic instrumentation for Java applications.

When an application is deployed with the corresponding instrumentation annotation, the OpenTelemetry Operator automatically injects the Java agent into the application container.



The `always_on` sampler captures every trace generated by the application, making it ideal for training and development environments. Production environments typically use sampling strategies to reduce telemetry volume.

Verify Automatic Instrumentation

Verify that the OpenTelemetry Collector is ready to receive telemetry data from instrumented applications.

View the collector logs.

```
oc logs deployment/otel-collector-collector \
  -n etx-app-dev
```

Initially, the collector logs may not contain any trace data because no instrumented applications have been deployed yet.



In a later lab, you'll deploy the ETX App Base application with automatic instrumentation enabled. As the application generates requests, you'll see distributed traces appear in the collector logs, confirming that telemetry is being

collected successfully.

Lab Summary

You installed the required operators, deployed the supporting infrastructure, and verified that each service is running and ready for use in the remaining labs.

You now have the following services available:

- **Red Hat OpenShift Pipelines** – Provides the Tekton-based CI/CD engine for building and deploying applications.
- **Red Hat AMQ Streams** – A single-node Apache Kafka cluster with the bootstrap endpoint `parasol-kafka-kafka-bootstrap:9092`.
- **PostgreSQL Database** – A PostgreSQL instance available at `parasol-db:5432` with the `parasol` database.
- **Red Hat Build of OpenTelemetry** – An OpenTelemetry Collector and automatic Java instrumentation configured for application observability.

The training application will use the following service endpoints during subsequent labs:

- **Kafka bootstrap server:** `parasol-kafka-kafka-bootstrap:9092`
- **PostgreSQL host:** `parasol-db:5432`
- **Database credentials:** Stored in the `parasol-db-secret` Secret

Red Hat Dev Spaces



What's in this section?

In this section, you prepare a complete cloud-native development environment using **Red Hat Openshift Dev Spaces** and GitLab.

You install and configure **Red Hat Openshift Dev Spaces**, integrate it with GitLab, create a reusable Devfile, and launch the **ETX App Base application** in a browser-based development workspace connected to the supporting platform services.

Dev Spaces Overview

In the previous sections, you learned how a **Software Factory** standardizes software delivery by providing developers with reusable services, automation, and engineering best practices.

One of the first capabilities provided by a Software Factory is a standardized development environment.

Instead of asking every developer to install IDEs, programming language runtimes, SDKs, CLI tools, and dependencies on their own workstation, organizations provide ready-to-use development environments that developers can access on demand.

This capability is provided by **Red Hat OpenShift Dev Spaces**.



Think about how long it takes today for a new developer to prepare their laptop before writing their first line of code. How much of that effort could be eliminated with a preconfigured development environment?

The Challenge

Setting up a local development environment is often one of the most time-consuming parts of developer onboarding.

A typical setup includes:

- Installing an IDE.
- Installing programming language runtimes.
- Downloading SDKs and command-line tools.

- Configuring Kubernetes access.
- Installing extensions and plugins.
- Managing project dependencies.
- Matching software versions used by the rest of the team.

Even after completing this setup, developers frequently encounter environment differences that lead to the familiar problem:

> "It works on my machine."

These inconsistencies slow onboarding, complicate collaboration, and introduce avoidable deployment issues.



When every developer maintains a different workstation configuration, development environments gradually drift apart, making troubleshooting, collaboration, and onboarding increasingly difficult.

What is OpenShift Dev Spaces?

OpenShift Dev Spaces provides **Cloud Development Environments (CDEs)** that run directly on OpenShift. Dev Spaces provides several capabilities that improve the developer experience by delivering consistent, cloud-based development environments.

Browser-Based IDE

Provides browser-based development workspaces that run in containers on OpenShift, allowing developers to code without installing local development tools.

Configuration as Code

Workspace configurations are defined as code, making development environments consistent, shareable, and easy to reproduce while reducing onboarding time.

Multiple Workspaces

Developers can create and manage multiple workspaces tailored to different applications, reducing context switching and improving productivity.

Instead of developing on a local workstation, developers use browser-based workspaces that are already configured with the required IDE, programming language runtimes, tools, dependencies, and cluster access.

Each workspace runs as containers on the OpenShift cluster while developers access it through a web browser using a familiar development environment.

This allows every developer to work with a consistent and reproducible environment regardless of the machine they are using.

Why Cloud Development Environments?

Cloud Development Environments improve both developer productivity and platform consistency.

Capability	Benefit
Rapid onboarding	Developers begin coding within minutes instead of spending hours preparing their workstation.
Consistent environments	Every developer uses the same tools, runtime versions, and dependencies.
Cloud-native development	Applications are developed in an environment that closely matches production.
Centralized security	Source code and development environments remain inside the organization's OpenShift platform.
Platform management	Development environments are centrally maintained by Platform Engineers.

Dev Spaces in the Software Factory

Within a Software Factory, Dev Spaces provides the standardized environment where application development begins.

A typical workflow looks like this:

1. Launch a development workspace.
2. Clone or open the application source code.
3. Develop and test application features.
4. Commit changes to Git.
5. Trigger the automated software delivery pipeline.
6. Allow the Software Factory to build, validate, and deploy the application.

Developers focus on writing application code while the Software Factory handles the surrounding automation and platform operations.

Benefits

For developers:

- Start coding immediately without local setup.
- Access projects from any machine using only a browser.
- Work in consistent environments across development teams.
- Easily switch between projects without reinstalling tools.

For Platform Engineers:

- Standardize development environments.
- Simplify developer onboarding.
- Improve security by centralizing development resources.
- Reduce environment-related support requests.



OpenShift Dev Spaces standardizes where developers build software, allowing Platform Engineers to centrally manage development environments while developers focus on delivering application features.

Reflection

▼ *Think about your own organization (Click to expand)*



- How long does it take to prepare a new developer workstation?
- Which tools must developers install manually?
- How often do environment differences cause development or deployment issues?
- How could centralized development environments improve developer productivity?

```
<div class="key-card">
```

Key Takeaway

OpenShift DevSpaces provides standardized Cloud Development Environments that eliminate workstation setup, improve consistency, and enable developers to start building applications quickly within the Software Factory.

```
</div>
```

Dev Spaces Architecture

In the previous lesson, you learned how OpenShift Dev Spaces provides standardized Cloud Development Environments for developers.

In this lesson, you'll look behind the scenes to understand how Dev Spaces creates and manages developer workspaces on OpenShift.

How a Workspace is Created

When a developer creates a new workspace, Dev Spaces follows a simple workflow:

1. The developer selects or imports a project that contains a **Devfile**.
2. The Dev Spaces Dashboard sends the workspace request.

3. The Dev Spaces Server reads the Devfile definition.
4. A **DevWorkspace** resource is created on OpenShift.
5. OpenShift provisions the required workspace containers.
6. The developer connects to the running workspace through the browser.

This process ensures every workspace is created from the same version-controlled configuration, providing a consistent development environment for every developer.

Core Components

The architecture consists of several services that work together to provision and manage developer workspaces.

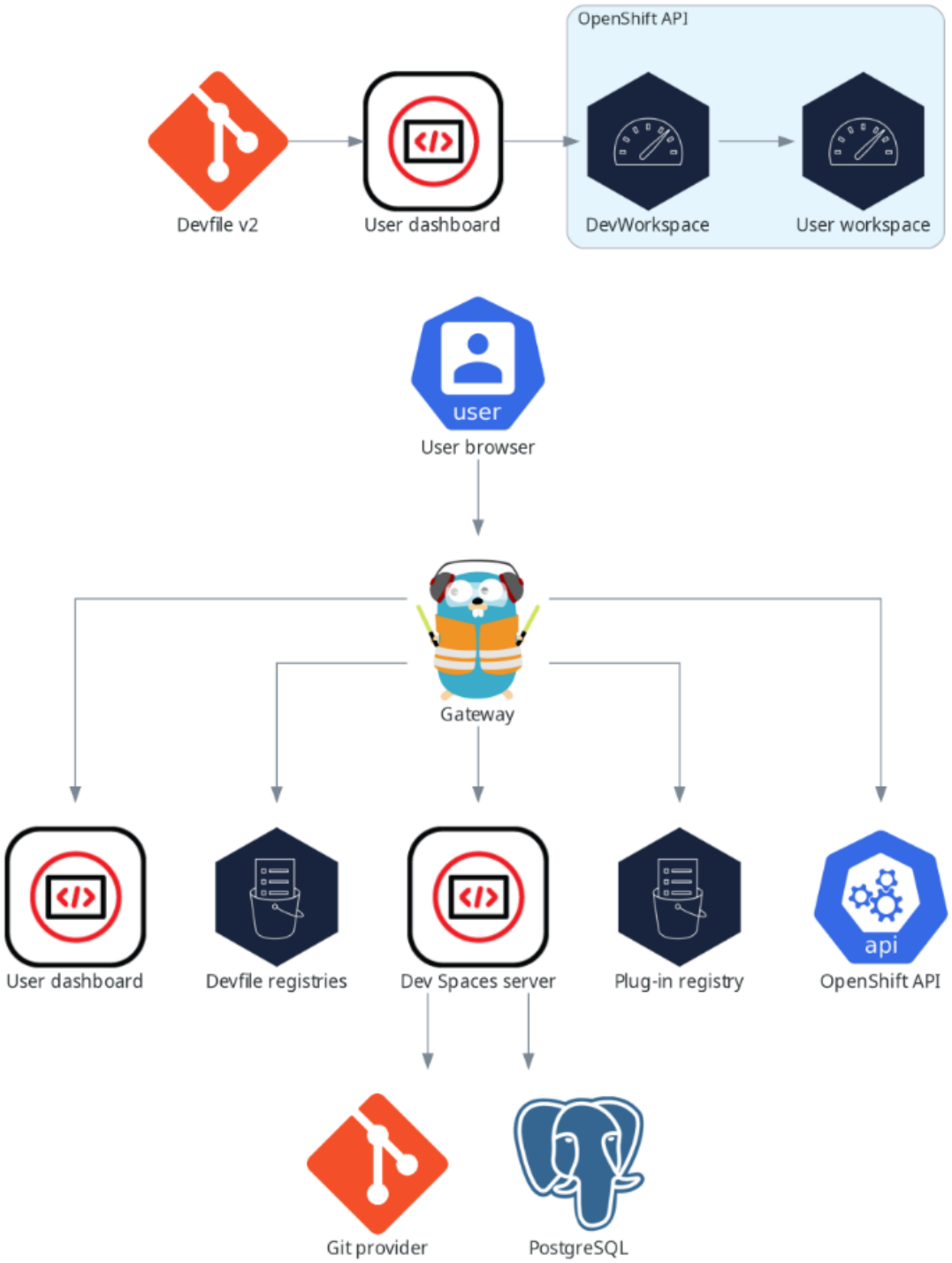


Figure 24. DevSpaces Architecture(Click the image to view it in full size in a new browser tab)

Component	Purpose
Devfile Registry	Stores reusable Devfile templates that define workspace configurations and sample development environments.
Plugin Registry	Provides IDE extensions and plugins that are installed into developer workspaces.
Dev Spaces Dashboard	The browser-based interface where developers create, start, stop, and manage their workspaces.

Component	Purpose
Dev Spaces Server	The core service that processes workspace requests, interprets Devfiles, and coordinates workspace creation.
Gateway	Routes user requests to the appropriate Dev Spaces services and workspace pods. It also handles authentication and authorization using OpenShift OAuth and Role-Based Access Control (RBAC).
PostgreSQL	Stores configuration data and workspace metadata used by the Dev Spaces platform.
OpenShift API	Creates and manages the Kubernetes resources required for each developer workspace.
DevWorkspace	A Kubernetes custom resource that represents an individual development workspace running on OpenShift.

How Dev Spaces Fits the Software Factory

Within the Software Factory, Dev Spaces provides the standardized development environment where application development begins.

Platform Engineers maintain the Dev Spaces platform, while developers simply launch a workspace and begin writing code.

Because workspaces are defined as code and provisioned automatically, organizations gain:

- Consistent development environments
- Faster developer onboarding
- Simplified platform management
- Improved security and governance



Most developers interact only with the Dev Spaces Dashboard. The remaining components operate behind the scenes to automatically provision, secure, and manage development workspaces on OpenShift.

Key Takeaway

OpenShift Dev Spaces combines Devfiles, platform services, and OpenShift to automatically provision standardized development workspaces, allowing developers to focus on building applications instead of configuring their development environment.

Lab 7: Install Red Hat Dev Spaces

In this lab, you'll install the Red Hat Dev Spaces Operator from OperatorHub and verify that it has been installed successfully.

By the end of this lab, you will have:

- Installed the Red Hat Dev Spaces Operator
- Verified that the operator installed successfully

Install the Red Hat Dev Spaces Operator

Install the Red Hat Dev Spaces Operator from OperatorHub. This operator manages the lifecycle of Dev Spaces and the developer workspaces that you'll create later in this training.

1. Open the OpenShift web console.
2. Sign in using the `admin` credentials provided on the RHDP Lab Environment page.
3. From the navigation menu, select **[Ecosystem]** → **[Software Catalog]**.

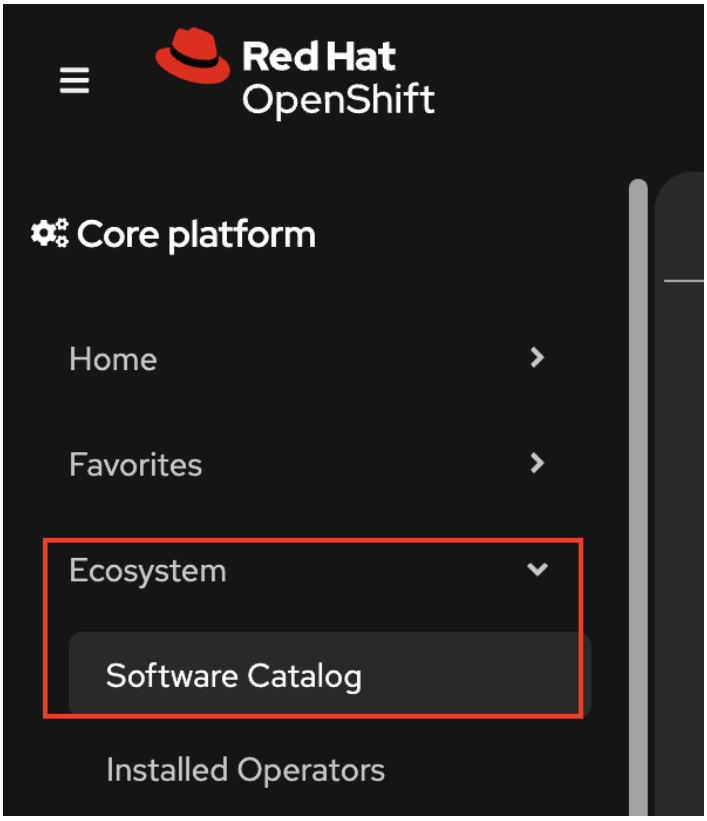


Figure 25. Ecosystem → Software Catalog(Click the image to view it in full size in a new browser tab)

4. Search for **Red Hat OpenShift Dev Spaces**.

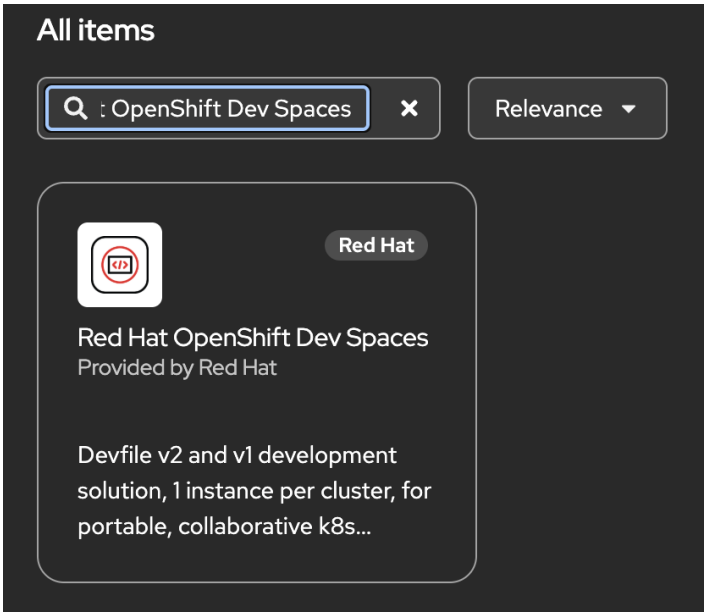


Figure 26. Red Hat Openshift DevSpaces Operator(Click the image to view it in full size in a new browser tab)

5. Select the operator and click **[Install]**.
6. Accept the default installation settings:
    - **Installation mode:** `All namespaces`
    - **Installed namespace:** `openshift-operators`
    - **Approval strategy:** `Automatic`

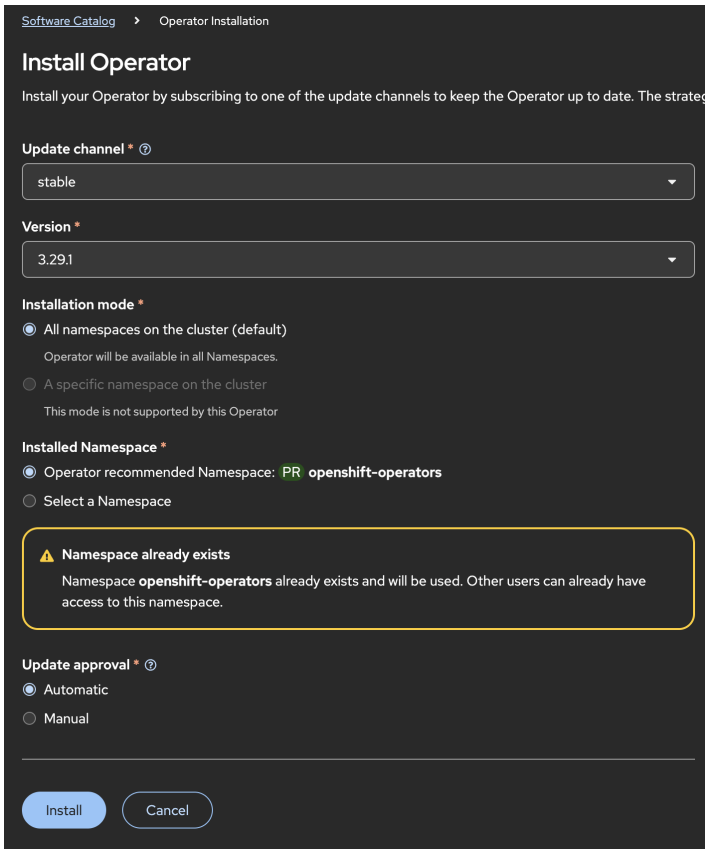


Figure 27. Install Red Hat Openshift DevSpaces(Click the image to view it in full size in a new browser tab)

7. Click **[Install]**.
8. Wait until the operator status changes to **Succeeded**.

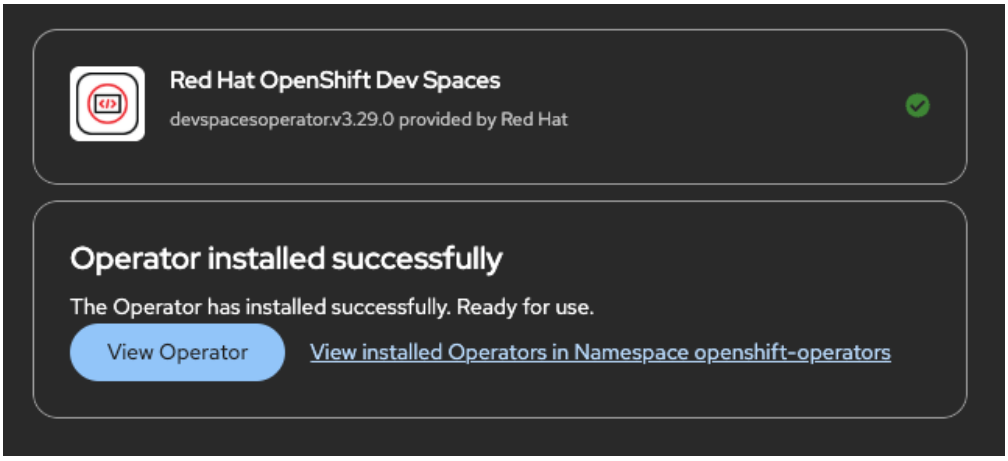


Figure 28. Dev Spaces Operator Installed(Click the image to view it in full size in a new browser tab)

Installing the Red Hat Openshift Dev Spaces Operator also installs the DevWorkspace Operator, which manages the lifecycle of developer workspaces.

Verify the Operator Installation

Confirm that the Red Hat Dev Spaces Operator has been installed successfully.

```
oc get csv -n openshift-operators | grep devspaces
```

Verify that the command returns the Dev Spaces ClusterServiceVersion (CSV) with a status of `Succeeded`.

A similar output should be displayed:

```
devspacesoperator.v3.27.1   Red Hat OpenShift Dev Spaces   3.27.1
devspacesoperator.v3.27.0   Succeeded
```

The operator runs from the `openshift-operators` namespace. In the next lab, you'll create a `CheCluster` resource, which provisions the `openshift-devspaces` namespace along with the Dev Spaces server components required to host developer workspaces.

Configure Dev Spaces

CheCluster

The `CheCluster` custom resource is the single configuration object for your Dev Spaces deployment. This is the sample configuration to create an instance with sensible defaults for this workshop environment:

```
apiVersion: org.eclipse.che/v2
kind: CheCluster
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false # ①
    metrics:
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org" # ②
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1 # ③
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false # ④
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace # ⑤
```

① Image puller pre-caches images on nodes to speed up workspace startup. Disable for small clusters; enable if startup time matters.

② Points at the public Open VSX registry so developers can install any extension. The default embedded registry has a very limited set.

③ Set to `-1` to disable run-time idling (useful during workshops). In production, set a reasonable value.

④ Required for nested containers (Podman-in-workspace). Must be `false`.

⑤ `per-workspace` gives each workspace its own PVC. This supports multiple workspaces per user without conflicts and provides clean isolation. Alternative: `per-user` shares a single PVC across all workspaces.

Lab 8: Configure Red Hat Dev Spaces

In this lab, you'll configure Red Hat Dev Spaces by deploying a `CheCluster` instance and verifying that the Dev Spaces dashboard is available.

A `CheCluster` defines the Dev Spaces server components, workspace configuration, authentication, and resource limits. Once deployed, it provides browser-based development environments for users.

By the end of this lab, you will have:

- Deployed a `CheCluster` instance
- Verified that the Dev Spaces platform is running
- Accessed the Dev Spaces dashboard
- Confirmed that OpenShift authentication is working

Deploy the Dev Spaces Instance

Begin by creating the namespace that will host the Dev Spaces components.

```
oc create namespace openshift-devspaces
```

Next, deploy the `CheCluster` resource.

Apply the following manifest.

```
oc apply -f - <<'EOF'
apiVersion: org.eclipse.che/v2
kind: CheCluster
metadata:
  name: devspaces
  namespace: openshift-devspaces
spec:
  components:
    cheServer:
      debug: false
      logLevel: INFO
    dashboard: {}
    devWorkspace: {}
    devfileRegistry: {}
    imagePuller:
      enable: false
    metrics:
```

```
      enable: true
    pluginRegistry:
      openVSXURL: "https://open-vsx.org"
  containerRegistry: {}
  devEnvironments:
    startTimeoutSeconds: 600
    secondsOfRunBeforeIdling: -1
    secondsOfInactivityBeforeIdling: 1800
    maxNumberOfWorkspacesPerUser: 5
    maxNumberOfRunningWorkspacesPerUser: 2
    defaultContainerResources:
      limits:
        cpu: "2"
        memory: 6Gi
      requests:
        cpu: 500m
        memory: 1Gi
    disableContainerRunCapabilities: false
    defaultEditor: che-incubator/che-code/latest
    defaultNamespace:
      autoProvision: true
      template: <username>-devspaces
    storage:
      pvcStrategy: per-workspace
EOF
```



Red Hat Openshift Dev Spaces uses OpenShift OAuth for authentication by default. Users sign in with their OpenShift credentials, and no external identity provider configuration is required.

Verify the Dev Spaces Deployment

Wait for the `CheCluster` resource to be created.

```
oc get checluster -n openshift-devspaces
```

Verify that a `CheCluster` named `devspaces` has been created.

Next, monitor the Dev Spaces pods until they are fully started.

```
oc get pods -n openshift-devspaces -w
```

Wait until all pods are in the `Running` state and each pod reports the expected number of ready containers (for example, `1/1` or `4/4`).

Finally, verify that the `CheCluster` is active.

```
oc get checluster devspaces \
  -n openshift-devspaces \
  -o jsonpath='{.status.chePhase}{"\n"}'
```

Verify that the command returns:

```
Active
```



Do not proceed until the `CheCluster` status is `Active`.

Access the Dev Spaces Dashboard

Once the `CheCluster` is active, retrieve the Dev Spaces dashboard URL.

```
oc get checluster devspaces \
  -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open the URL in your web browser. When prompted, sign in using your OpenShift credentials provided in RHDP Lab Environment page (for example, `admin` and the cluster administrator password).

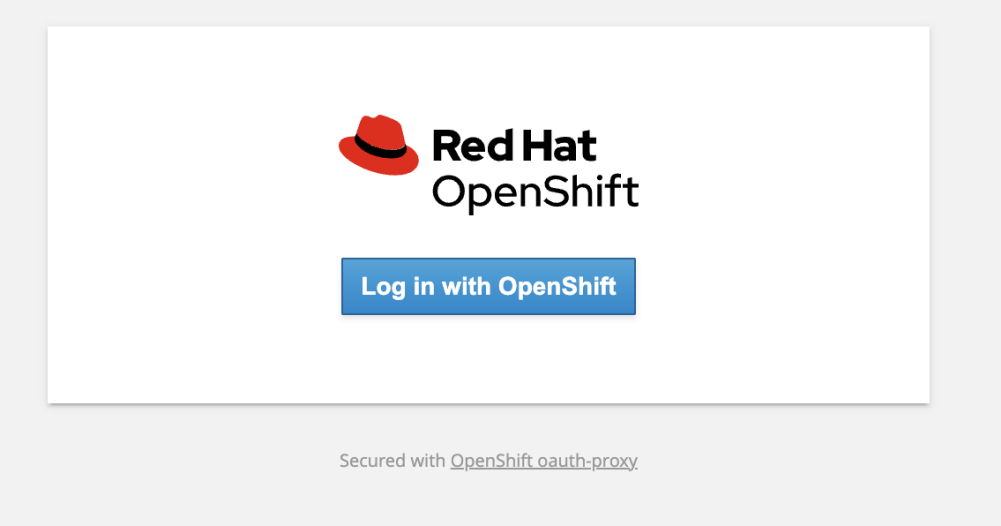


Figure 29. Red Hat Openshift Login(Click the image to view it in full size in a new browser tab)

If prompted to authorize access, click **[Allow selected permissions]**.

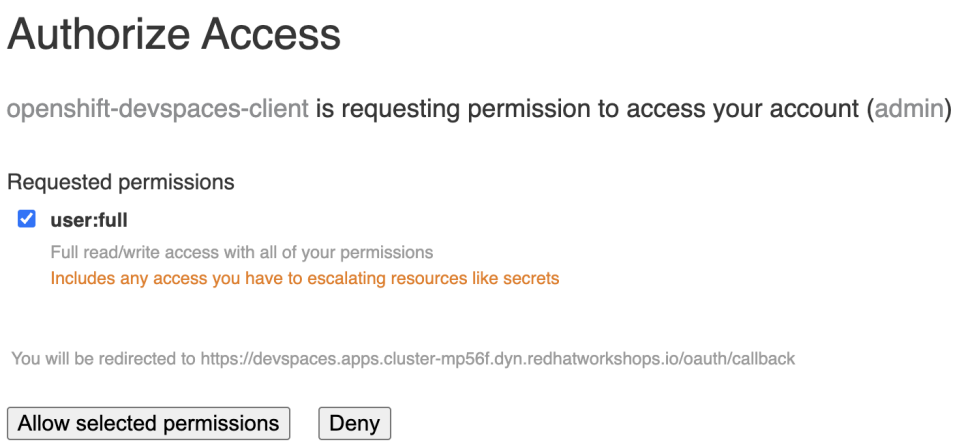


Figure 30. Authorize Dev Spaces Access(Click the image to view it in full size in a new browser tab)

After authentication completes, the Red Hat Openshift Dev Spaces home page is displayed.

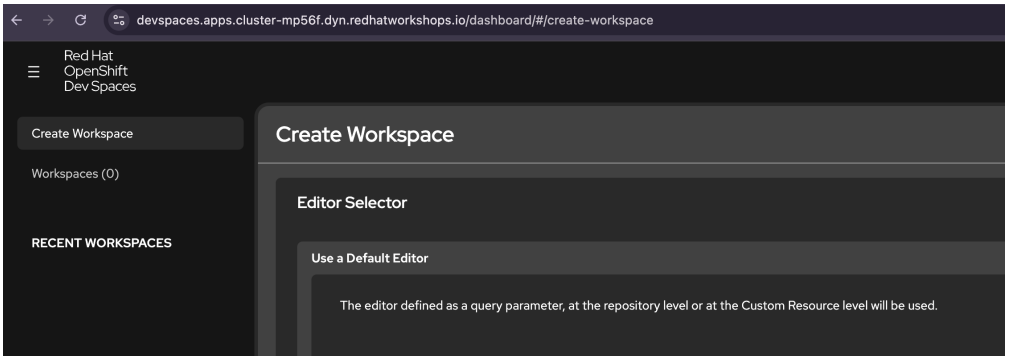


Figure 31. Dev Spaces Home Page(Click the image to view it in full size in a new browser tab)



You can also access Red Hat Openshift Dev Spaces directly from the OpenShift web console by clicking the Application Launcher in the upper-right corner and selecting **Red Hat OpenShift Dev Spaces**. You'll be automatically authenticated using your existing OpenShift session.

Appendix: Administrator Reference Commands

The following commands are provided for reference and are commonly used by cluster administrators to modify Dev Spaces configuration.



Do not run these commands during the training unless you're instructed to change the Dev Spaces configuration.

```
# Change storage strategy
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p '{"spec":{"devEnvironments":{"storage":{"pvcStrategy":"per-workspace"}}}}'

# Change maximum workspaces per user
oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
```

```
  -p '{"spec":{"devEnvironments":{"maxNumberOfWorkspacesPerUser":10}}}'

# View the current storage strategy
oc get checluster devspaces -n openshift-devspaces \
  -o jsonpath='{.spec.devEnvironments.storage.pvcStrategy}{"\n"}'
```

Creating a Workspace

Now let's see what actually happens when a developer signs in and creates a development workspace.

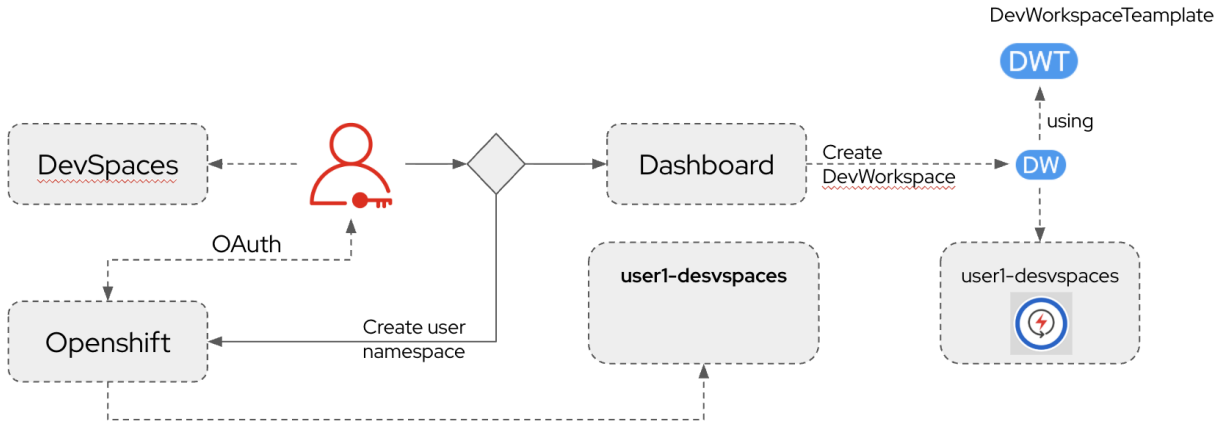


Figure 32. Workspace Creation Workflow(Click the image to view it in full size in a new browser tab)

Workspace Creation Workflow

Creating a workspace is designed to be a simple self-service experience.

Behind the scenes, Red Hat Openshift Dev Spaces automatically provisions the required OpenShift resources so developers can begin coding within minutes.

The workflow consists of the following steps:

1. The developer signs in to the Red Hat Openshift Dev Spaces Dashboard using their OpenShift credentials.
2. Red Hat Openshift Dev Spaces authenticates the user through OpenShift OAuth.
3. If this is the user's first login, OpenShift automatically creates a dedicated namespace named `<username>-devspaces`.
4. The developer creates either:
   - An empty workspace, or
   - A workspace based on a predefined template.
5. Red Hat Openshift Dev Spaces creates a **DevWorkspace** resource.
6. The DevWorkspace references a **DevWorkspaceTemplate**, which defines the containers, tools, storage, and runtime configuration required for the workspace.

7. OpenShift provisions the workspace pod.
8. The developer connects to the running workspace through their browser and begins coding.

Authentication

By default, OpenShift Red Hat Openshift Dev Spaces integrates directly with the OpenShift authentication system.

This provides a seamless sign-in experience using existing cluster credentials.

Key points:

- Uses OpenShift OAuth by default.
- Supports any OpenID Connect (OIDC) identity provider.
- Can integrate with Red Hat Single Sign-On (RHSSO) or Red Hat Build of Keycloak.
- Uses OpenShift Role-Based Access Control (RBAC) to control access to workspaces.

Because authentication is delegated to OpenShift, developers don't need separate Red Hat Openshift Dev Spaces accounts.

Automatic Workspace Isolation

Every developer receives an isolated workspace namespace.

For example:

```
alice-devspaces
bob-devspaces
charlie-devspaces
```

Each namespace contains only that developer's workspaces and resources.

This provides:

- Resource isolation
- Better security
- Simplified administration
- Multi-tenant development environments

Platform Engineers can centrally manage resource quotas and permissions while developers remain isolated from one another.

Workspace Templates

Developers don't need to manually configure development environments.

Instead, they create workspaces from reusable templates.

A template typically defines:

- Development container image
- IDE configuration
- Required tools and runtimes
- Persistent storage
- Environment variables
- Startup commands

These templates ensure every developer starts from a consistent environment that follows organizational standards.



Developers simply choose a template and start coding. Red Hat Openshift Dev Spaces automatically provisions the required OpenShift resources, creates the development workspace, and provides browser-based access to a fully configured environment.

Key Takeaway

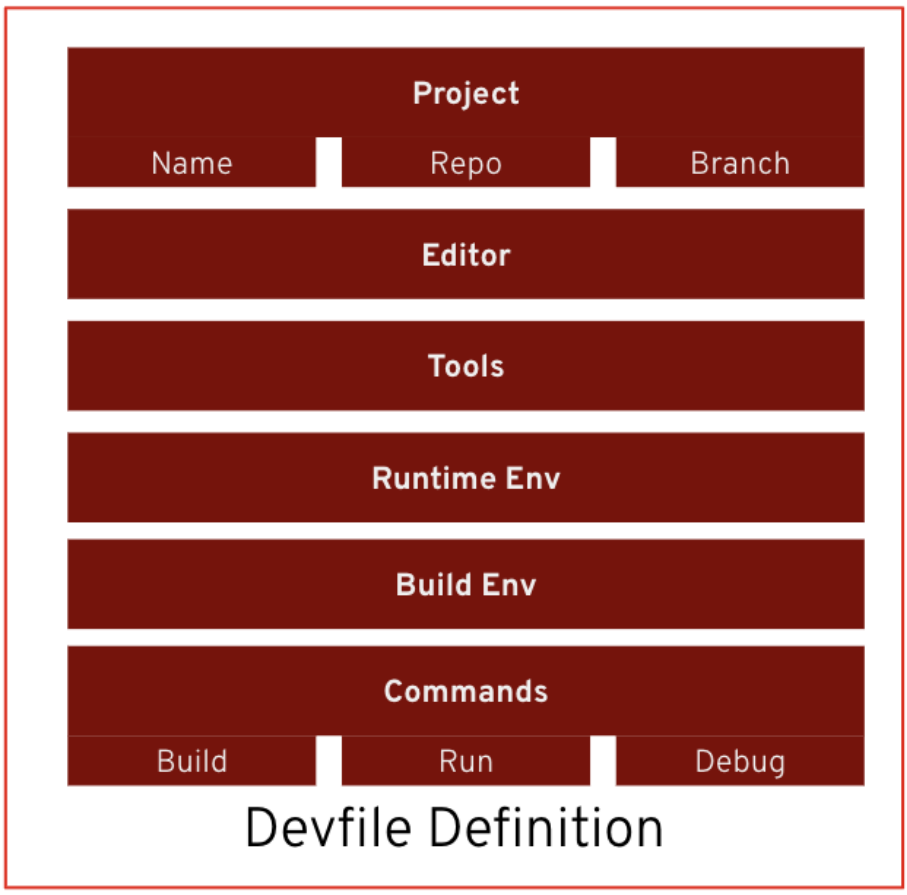
Red Hat OpenShift Red Hat Openshift Dev Spaces automates workspace provisioning by authenticating users through OpenShift, creating isolated namespaces, and provisioning standardized development environments from reusable workspace templates.

Devfiles

In the previous lesson, you learned how OpenShift Dev Spaces provisions developer workspaces automatically.

But how does Dev Spaces know which tools, runtimes, and commands should be available inside each workspace?

The answer is a **Devfile**.



What is a Devfile?

A Devfile is a YAML file, typically named `devfile.yaml`, stored in the root of a Git repository.

Platform Engineers use Devfiles to describe and standardize development environments, allowing every developer working on a project to use the same tools, runtime, and workspace configuration.

Instead of manually configuring development environments, Dev Spaces simply reads the Devfile and provisions the workspace automatically.



A Devfile acts as the blueprint for a development workspace. It defines everything needed to create a consistent development environment.

What Does a Devfile Define?

A Devfile can describe every aspect of a development workspace, including:

- Project metadata such as the repository and default branch
- The development editor and IDE configuration
- Container images and runtime environment
- Development tools and dependencies
- Build and runtime environments
- Volumes and persistent storage

- Build, run, debug, and test commands

Because the Devfile is version-controlled alongside the application source code, every developer receives the same development environment regardless of their local machine.

Why Devfiles Matter

Without Devfiles, every developer must configure their own workstation.

This often leads to inconsistent tool versions, missing dependencies, and the classic problem:

> "It works on my machine."

Devfiles eliminate this by defining the workspace as code.

Benefits include:

- Consistent development environments
- Faster developer onboarding
- Reproducible workspaces
- Version-controlled environment definitions
- Simplified collaboration across teams

Devfile Structure

A Devfile is written in YAML and is organized into several logical sections.

```
schemaVersion: 2.2.0
metadata:
  name: my-project

attributes: {}   # IDE settings
components: []   # Containers, volumes, endpoints
commands: []     # Build, run, debug commands
events: {}       # Lifecycle hooks
```

The most common sections are:

Section	Purpose
Metadata	Identifies the workspace and project.
Attributes	Stores IDE-specific configuration, such as recommended extensions.
Components	Defines containers, volumes, endpoints, and other workspace resources.
Commands	Specifies build, run, test, and debug commands available from the IDE.
Events	Executes lifecycle actions such as `preStart` and `postStart`.

Devfiles and the Universal Developer Image

A Devfile is recommended, but it is not required to create a workspace.

If a Git repository does not contain a `devfile.yaml`, OpenShift Dev Spaces automatically provisions a workspace using the **Universal Developer Image (UDI)**.

The Universal Developer Image includes many commonly used development tools and programming language runtimes, allowing developers to start coding immediately.

As projects become more specialized, teams typically provide their own Devfile to customize the workspace with project-specific tools, dependencies, and commands.

How Devfiles Fit the Software Factory

Within the Software Factory, Platform Engineers maintain Devfiles as reusable definitions for development environments.

Developers simply clone a repository or create a new workspace, and Dev Spaces provisions the environment automatically.

This ensures every project follows organizational standards while reducing setup time and operational overhead.



Devfiles define development environments as code, allowing organizations to deliver standardized, repeatable, and version-controlled workspaces across teams.

Key Takeaway

A **Devfile** is the blueprint for a Dev Spaces workspace. It defines the tools, containers, commands, and runtime configuration required for development, enabling consistent and reproducible environments for every developer.

Lab 9: GitLab Integration and Shared Secrets

In this lab, you'll integrate Red Hat Dev Spaces with GitLab using OAuth and configure shared secrets that are automatically available to developer workspaces.

GitLab integration enables developers to clone, pull, and push repositories without manually configuring Git credentials. You'll also configure shared workspace secrets that can be used to provide common configuration, API keys, and service credentials to every workspace.

By the end of this lab, you will have:

- Configured GitLab OAuth integration for Red Hat Dev Spaces
- Registered GitLab as a source code management provider

- Created shared secrets for developer workspaces

Configure GitLab OAuth Integration

Red Hat Dev Spaces integrates with GitLab using OAuth. Once configured, users authenticate with their GitLab account and can access repositories directly from their workspaces.

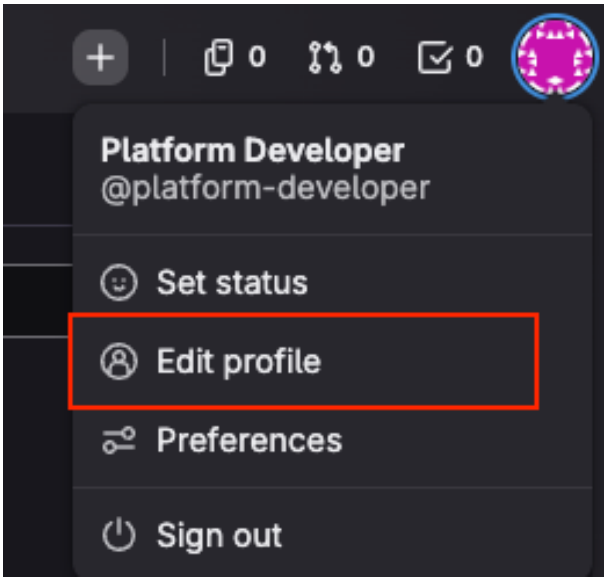
Create a GitLab OAuth Application

Create an OAuth application in GitLab that Red Hat Dev Spaces will use for authentication.

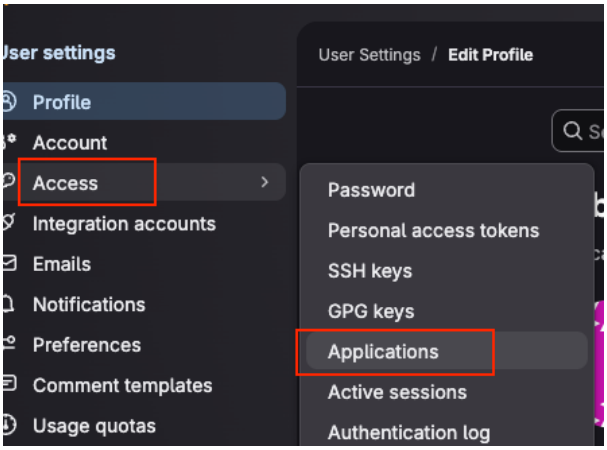
1. Execute below command in Openshift cluster to retrieve the **openshift cluster ingress domain** route from your OpenShift cluster and store it in a variable for use in next steps.

```
OPENSHIFT_CLUSTER_INGRESS_DOMAIN=$(oc get ingresses.config.openshift.io cluster -o
jsonpath='{.spec.domain}')
echo ${OPENSHIFT_CLUSTER_INGRESS_DOMAIN}
```

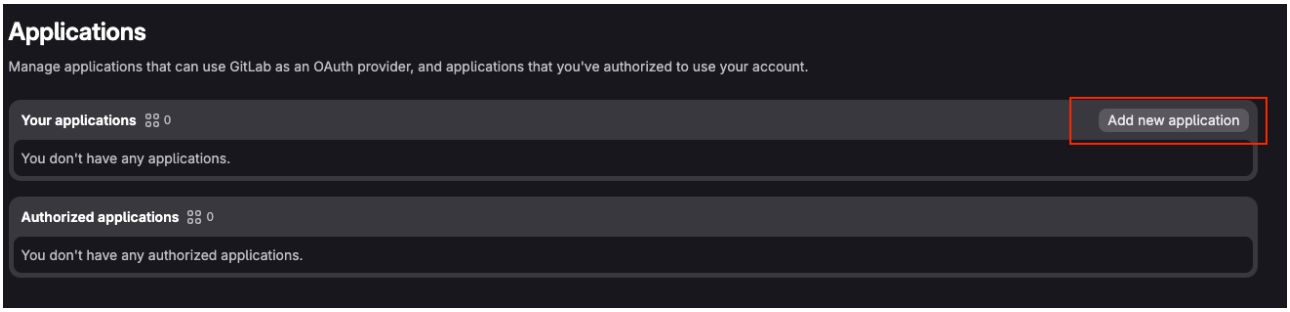
2. Navigate to the **GitLab URL** provided on your RHDP lab information page.
3. Sign-in using **ETX SSO** using credentials `username: platform-developer` / `password: openshift` if you have not logged in yet. After successful authentication, you'll be redirected back to GitLab.
4. Click your avatar in the upper-right corner and select **[Edit Profile]**.



5. Navigate to **Access › Applications**.



6. Click **[Add new application]**.



7. Configure the application using the following settings, make sure to substitute the value for **OPENSHIFT_CLUSTER_INGRESS_DOMAIN** with the value your retrived in first step.

Name	`devspaces`
Redirect URI	`https://devspaces.OPENSHIFT_CLUSTER_INGRESS_DOMAIN/api/oauth/callback`
Confidential	Enabled
Scopes	`api`, `write_repository`, `openid`

8. Click **[Save application]**.
9. Record the **Application ID** and **Secret**. You'll use these values in the next step.

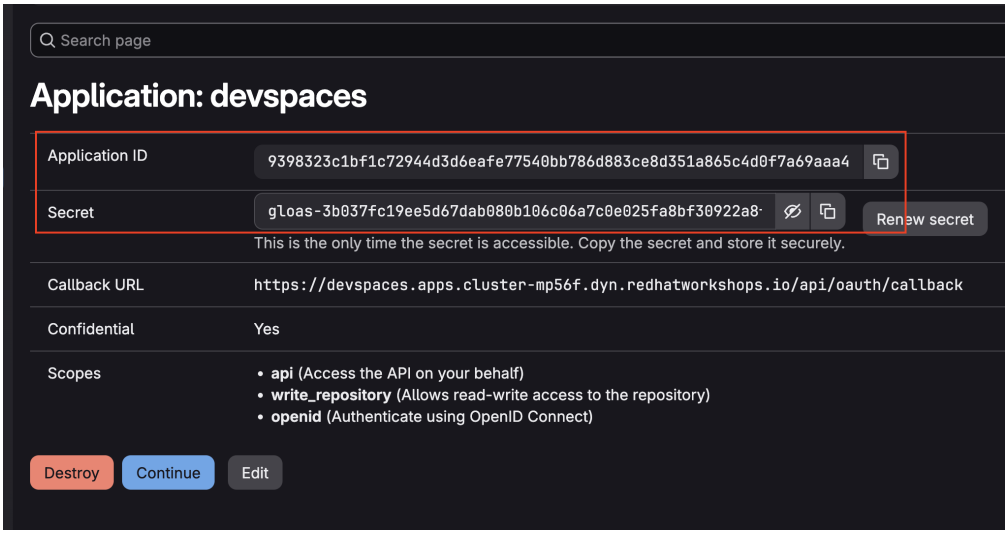


Figure 33. GitLab Oauth Application Credentials(Click the image to view it in full size in a new browser tab)

Configure GitLab Authentication

Replace `GITLAB_APPLICATION_ID` and `GITLAB_APPLICATION_SECRET` with the values generated by GitLab before running the following commands.

```
GITLAB_APPLICATION_ID=<add-your-value-here>
GITLAB_APPLICATION_SECRET=<add-your-value-here>
```

Create a Kubernetes Secret that stores the GitLab OAuth credentials. Red Hat Dev Spaces automatically discovers this secret based on its labels and annotations.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "Using GitLab URL: $GITLAB_URL"

oc create secret generic gitlab-oauth-config \
  -n openshift-devspaces \
  --from-literal=id="$GITLAB_APPLICATION_ID" \
  --from-literal=secret="$GITLAB_APPLICATION_SECRET"

oc annotate secret gitlab-oauth-config \
  -n openshift-devspaces \
  che.eclipse.org/oauth-scm-server=gitlab \
  che.eclipse.org/scm-server-endpoint="$GITLAB_URL"

oc label secret gitlab-oauth-config \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
  app.kubernetes.io/component=oauth-scm-configuration

echo "GitLab OAuth secret created successfully"
```

Next, register GitLab with the Dev Spaces `CheCluster`.

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

echo "Registering GitLab URL: $GITLAB_URL"

oc patch checluster devspaces -n openshift-devspaces \
  --type='merge' \
  -p "{\"spec\":{\"gitServices\":{\"gitlab\":[{\"secretName\":\"gitlab-oauth-
config\",\"endpoint\":\"$GITLAB_URL\"}]}}}"
```



After GitLab is registered, users are prompted to authorize Dev Spaces the first time they open a workspace that references a GitLab repository. Once authorized, Git operations such as clone, pull, and push use the user's GitLab identity automatically.

Configure Shared Workspace Secrets

Red Hat Openshift Dev Spaces can automatically mount Kubernetes Secrets and ConfigMaps into every workspace. This allows administrators to provide shared configuration, credentials, or API keys without requiring developers to configure them manually.

Secrets can be injected in one of the following ways:

- **Environment variables** (`mount-as: env`)
- **Files** (`mount-as: file`)
- **Individual files using subpaths** (`mount-as: subpath`)

Configure AI Coding Assistant Credentials

Configures shared credentials for an AI coding assistant such as **Continue**, Roo, or Cline. Begin by obtaining an API key from the Red Hat Demo Platform MaaS.

1. Open the Red Hat Demo Platform MaaS portal in your browser.

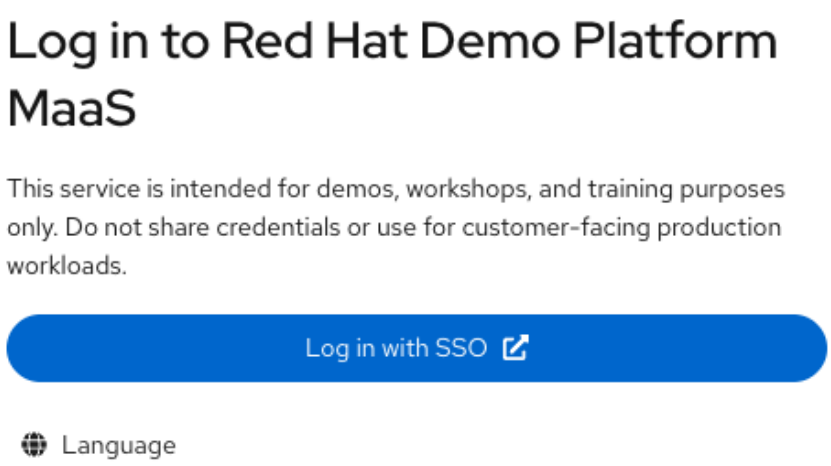


Figure 34. RHDP MaaS Login page(Click the image to view it in full size in a new browser tab)

2. Sign in using your **Red Hat Demo Platform [RHDP]** credentials.
3. Select **Model** option.

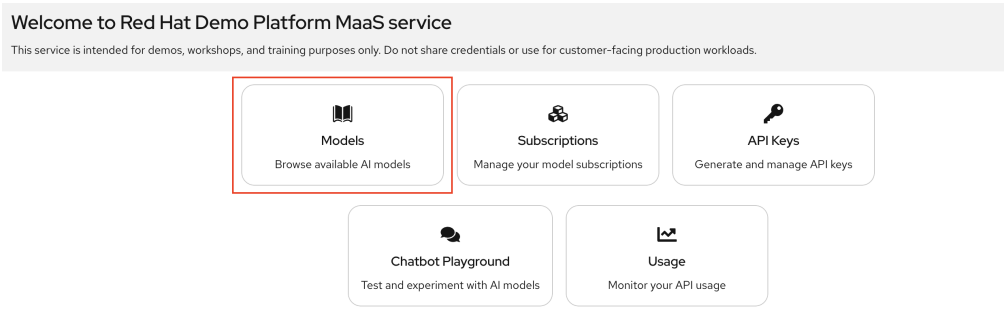


Figure 35. RHDP MaaS home page(Click the image to view it in full size in a new browser tab)

4. Search for `qwen3-14b` model and select it.

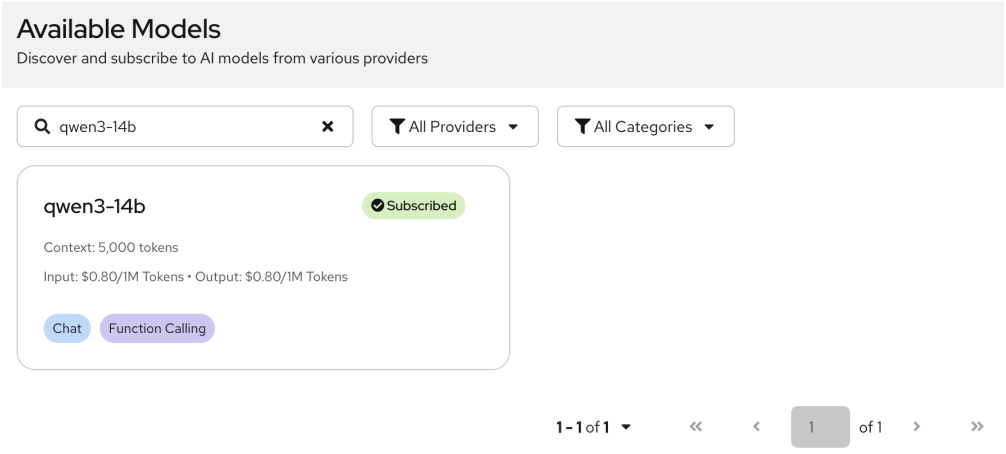


Figure 36. Model search result(Click the image to view it in full size in a new browser tab)

5. Subscribe to the model.

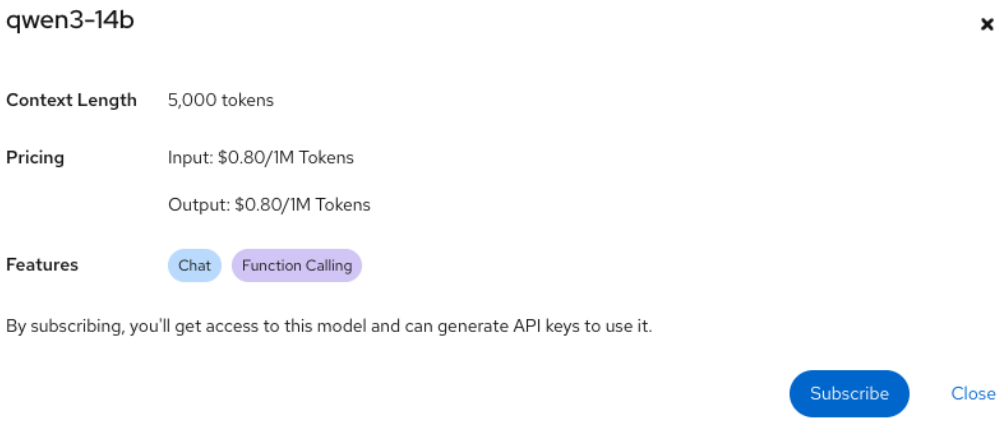


Figure 37. Subscribe the model(Click the image to view it in full size in a new browser tab)

6. Navigate back to RHDP MaaS home page and select **API Keys**.

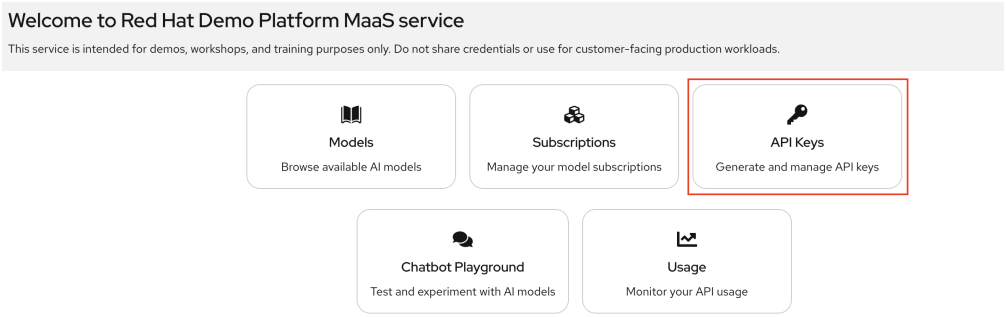


Figure 38. RHDP MaaS home page(Click the image to view it in full size in a new browser tab)

7. Create a new API key or select an existing one.



Figure 39. Create API key(Click the image to view it in full size in a new browser tab)

8. Copy the generated API key.
9. Create the shared secret.



Replace `REPLACE_API_KEY` with your MaaS API key before running the command.

```
oc create secret generic continue-llm-credentials \
  -n openshift-devspaces \
  --from-literal=LLM_API_KEY="REPLACE_API_KEY" \
  --from-literal=LLM_BASE_URL="https://maas-rhdp.apps.maas.redhatworkshops.io/v1"

oc annotate secret continue-llm-credentials \
  -n openshift-devspaces \
  controller.devfile.io/mount-as=env

oc label secret continue-llm-credentials \
  -n openshift-devspaces \
  app.kubernetes.io/part-of=che.eclipse.org \
```

```
  app.kubernetes.io/component=workspaces-config

echo "LLM credentials secret created successfully"
```



After the secret has been created, every newly created workspace automatically receives the following environment variables: `LLM_API_KEY`, `LLM_BASE_URL`



Later in this training, you'll create a Devfile that uses these environment variables to automatically configure AI coding assistants such as Continue.

Lab 10: Red Hat Openshift Dev Spaces Workspace

In this lab, you'll create a Red Hat Openshift Dev Spaces workspace for the **ETX App Base** application, configure it with a custom **Devfile**, and verify that it is ready for cloud-native application development.

The **Devfile** defines the development environment, including the container image, development commands, environment variables, and IDE extensions. By storing the Devfile in the Git repository, every developer can create an identical workspace with a single click.

By the end of this lab, you will have:

- Created a Red Hat Dev Spaces workspace
- Configured Git for source control
- Created and committed a `devfile.yaml`
- Recreated the workspace using the Devfile
- Verified the workspace configuration and connectivity to PostgreSQL and Kafka

Access the Dev Spaces Dashboard

Begin by accessing the Red Hat Dev Spaces dashboard.

Retrieve the dashboard URL.

```
oc get checluster devspaces \
  -n openshift-devspaces \
  -o jsonpath='{.status.cheURL}{"\n"}'
```

Open the URL in your web browser and sign in using your OpenShift credentials.



Red Hat Dev Spaces uses OpenShift OAuth for authentication. No additional login configuration is required.

Create an Initial Workspace

The `etx_app_base_app` repository does not yet contain a Devfile. In this section, you'll create a temporary workspace that you'll use to add the `devfile.yaml` to the repository.

1. From the Dev Spaces dashboard, click **[Create Workspace]** and select **Import from Git**.

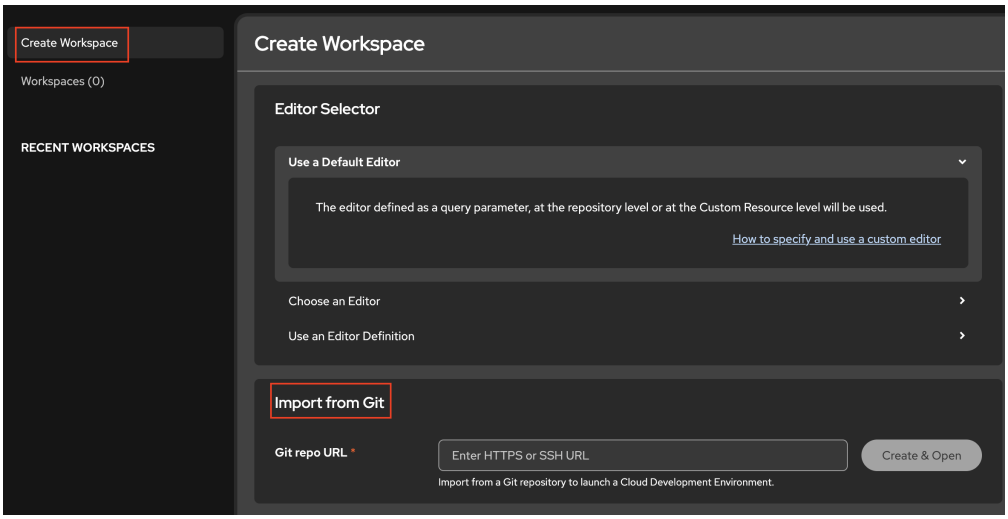


Figure 40. Red Hat Openshift Dev Spaces Home Page(Click the image to view it in full size in a new browser tab)

2. Retrieve the GitLab URL.

```
GITLAB_URL="https://$(oc get route \
  -n gitlab \
  -l app=webservice \
  -o jsonpath='{.items[0].spec.host}')"

echo "${GITLAB_URL}"
```

3. Paste the following repository URL into Dev Spaces.

```
{GITLAB_URL}/software-factory/etx_app_base_app.git
```



Figure 41. Sample Git URL(Click the image to view it in full size in a new browser tab)

4. Click **[Create & Open]**.
5. If prompted, authorize GitLab access using OAuth.

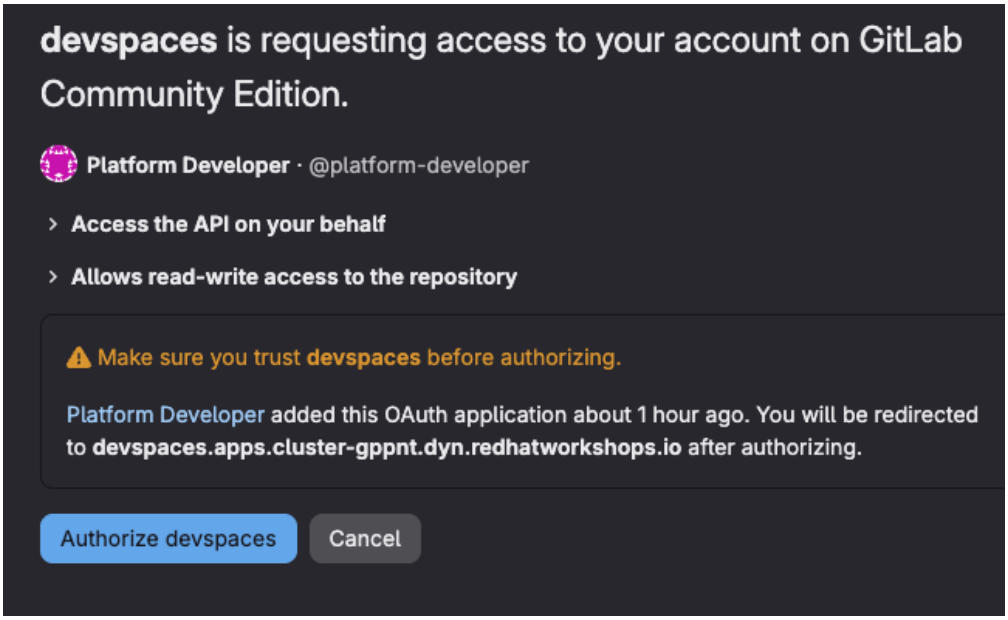


Figure 42. Authorize GitLab Access(Click the image to view it in full size in a new browser tab)



Because the repository does not yet contain a `devfile.yaml`, Dev Spaces automatically creates the workspace using the Universal Developer Image (UDI).

Configure Git inside your Workspaces

After the workspace opens, configure your Git identity.

Open a terminal by clicking **[hamburger icon]** and selecting **Terminal › New Terminal**.

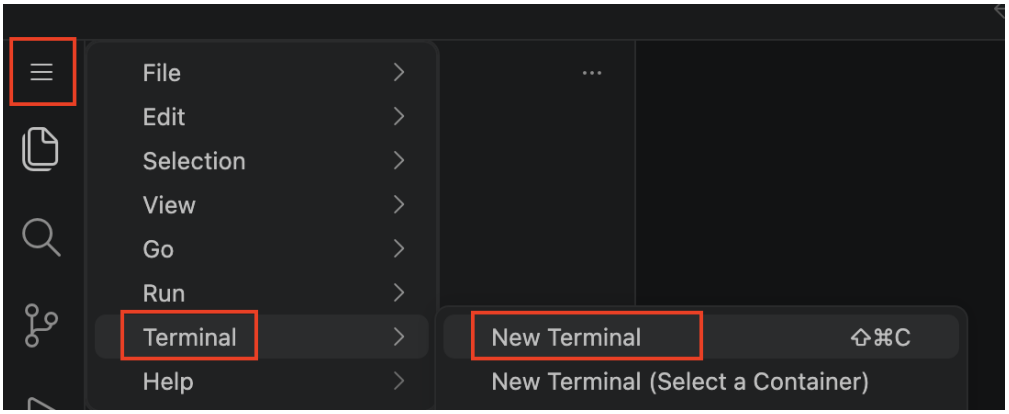


Figure 43. Terminal Option(Click the image to view it in full size in a new browser tab)

Run the following commands.

```
git config --global user.email "platform-developer@example.com"
git config --global user.name "Platform Developer"
```

Create a Devfile

The Devfile defines the development environment for the application, including the container image, development commands, environment variables, and IDE configuration.

Create the `devfile.yaml` file using the following command.



Make sure to execute below command inside the newly opened terminal in your workspaces.

```
cat > devfile.yaml <<'EOF'
schemaVersion: 2.2.0
metadata:
  name: etx-app-base-app
components:
  - name: development-tooling
    container:
      image: quay.io/devfile/universal-developer-image:ubi9-latest
      env:
        - name: QUARKUS_HTTP_HOST
          value: "0.0.0.0"
        - name: MAVEN_OPTS
          value: "-Dmaven.repo.local=/home/user/.m2/repository"
        # External service connections
        - name: POSTGRESQL_HOST
          value: "parasol-db.etx-app-dev.svc.cluster.local"
        - name: POSTGRESQL_DATABASE
          value: "parasol"
        - name: POSTGRESQL_USER
          value: "parasol"
        - name: POSTGRESQL_PASSWORD
          value: "parasol"
        - name: KAFKA_BOOTSTRAP_SERVERS
          value: "parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092"
      memoryLimit: 5Gi
      cpuLimit: 2500m
      volumeMounts:
        - name: m2
          path: /home/user/.m2
      endpoints:
        - name: quarkus-dev
          targetPort: 8080
          exposure: public
          protocol: https
        - name: debug
          targetPort: 5005
          exposure: none
  - name: m2
    volume:
      size: 1G
```

```
commands:
  - id: package
    exec:
      label: "1. Package the application"
      component: development-tooling
      commandLine: "./mvnw package"
      group:
        kind: build
        isDefault: true
  - id: start-dev
    exec:
      label: "2. Start Development mode (Hot reload + debug)"
      component: development-tooling
      commandLine: "./mvnw compile quarkus:dev"
      group:
        kind: run
        isDefault: true
  - id: init-continue
    exec:
      label: "Initialize Continue config"
      component: development-tooling
      workingDir: /home/user
      commandLine: |
        mkdir -p /home/user/.continue
        cat > /home/user/.continue/config.yaml << EOF
        name: Continue Config
        version: 0.0.1
        models:
          - name: qwen3-14b
            provider: vllm
            model: qwen3-14b
            apiKey: ${LLM_API_KEY}
            apiBase: ${LLM_BASE_URL}
            roles:
              - chat
        EOF
      group:
        kind: build
events:
  postStart:
    - init-continue
EOF
```

Verify that the file was created successfully.

```
ls -la devfile.yaml
head -20 devfile.yaml
```

Commit the Devfile

Commit the Devfile to a feature branch so it can be used when creating a new workspace.

```
git checkout -b feature/add-devfile

git add devfile.yaml
git commit -m "feat: add devfile for Dev Spaces configuration"

git push origin feature/add-devfile
```

Recreate the Workspace Using the Devfile

Now that the repository contains a Devfile, recreate the workspace so Dev Spaces can automatically configure the development environment.

Delete the Temporary Workspace

1. Return to the Dev Spaces dashboard.
2. Locate your workspace.

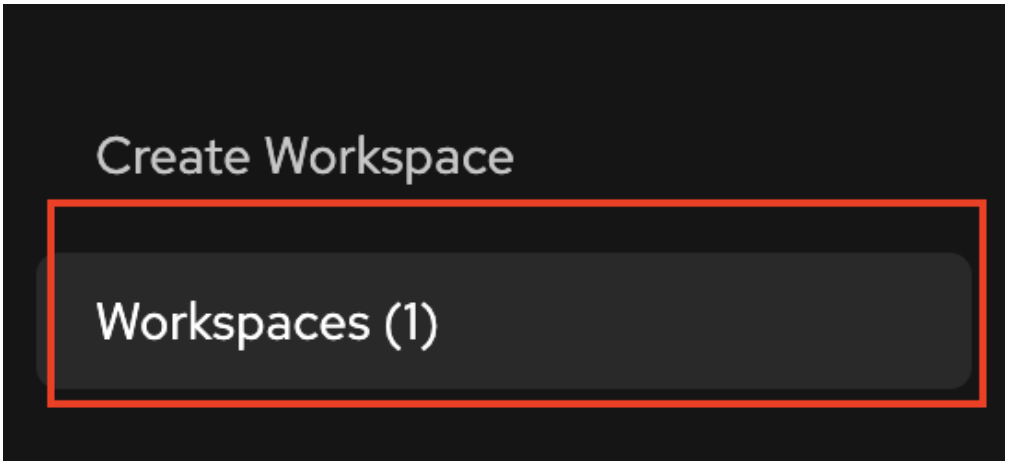


Figure 44. Workspace Menu(Click the image to view it in full size in a new browser tab)

1. Open the options menu and select **Delete Workspace**.

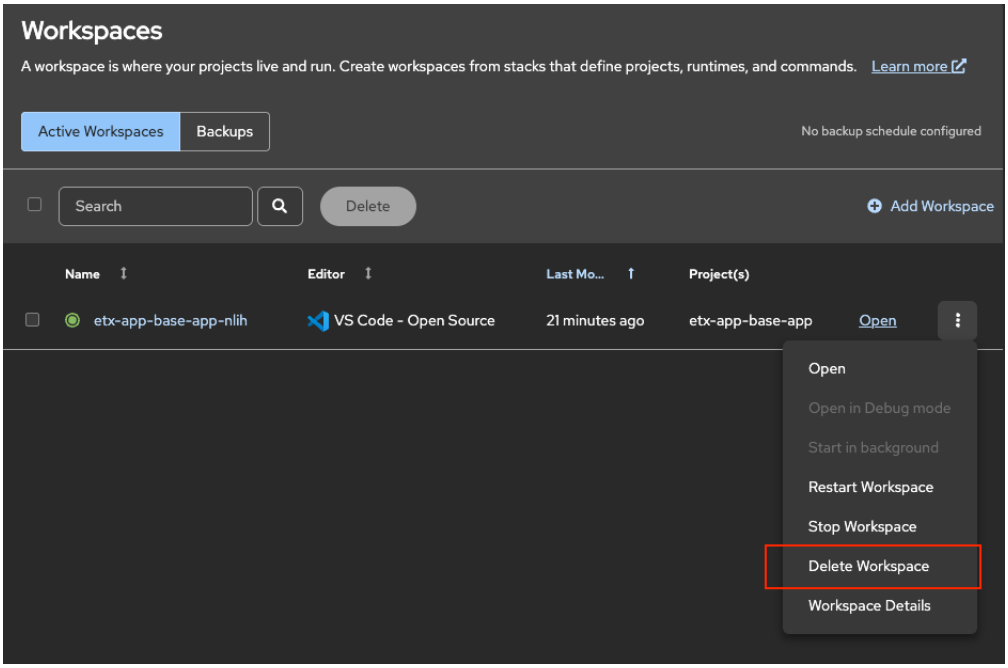


Figure 45. Delete Option(Click the image to view it in full size in a new browser tab)

1. Confirm the deletion.

Create a Workspace from the Devfile

You can create a workspace using either a direct URL or the Dev Spaces web interface.

Direct URL (Recommended)

Use a direct URL to create a workspace from a specific branch.

Get the required URLs:

```
# Get Dev Spaces URL
DEVSPACES_URL="https://$(oc get checluster devspaces -n openshift-devspaces -o
jsonpath='{.status.cheURL}' | sed 's|https://||')"

# Get GitLab URL
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Construct the direct workspace URL
echo "${DEVSPACES_URL}#${GITLAB_URL}/software-factory/etx_app_base_app/-
/tree/feature/add-devfile"
```

Open the generated URL in your browser. Dev Spaces will:

1. Automatically import the repository from the feature/add-devfile branch
2. Detect and use the devfile.yaml
3. Configure the workspace with all environment variables and settings

This skips the manual "Create Workspace → Import from Git" flow.

Manual UI

Create a workspace through the Dev Spaces dashboard.

1. Click **[Create Workspace]**
2. Select **Import from Git**
3. Paste the repository URL (use the actual GitLab URL from your cluster):

```
{gitlab_url}/software-factory/etx_app_base_app/-/tree/feature/add-devfile
```

4. Click **[Create & Open]**

When the workspace starts, Dev Spaces detects the `devfile.yaml` and automatically configures the development environment.

The workspace includes:

- PostgreSQL connection settings
- Kafka connection settings
- Persistent Maven repository cache
- Quarkus development commands
- Automatic AI assistant initialization



The direct workspace URL can be bookmarked and reused for future development sessions.

Verify the Workspace

After the workspace opens, verify that it has been configured correctly.

Verify the Repository

Open a terminal in your newly created workspace.

```
git branch --show-current
git log --oneline -3
ls -la devfile.yaml
```

Verify that:

- the current branch is `feature/add-devfile`
- `devfile.yaml` exists in the repository root

Verify Environment Variables

Confirm that the Devfile environment variables were applied.

```
echo "POSTGRESQL_HOST: $POSTGRESQL_HOST"
echo "POSTGRESQL_DATABASE: $POSTGRESQL_DATABASE"
echo "POSTGRESQL_USER: $POSTGRESQL_USER"
echo "KAFKA_BOOTSTRAP_SERVERS: $KAFKA_BOOTSTRAP_SERVERS"
```

The output should display the PostgreSQL and Kafka connection information.



If the environment variables are empty, recreate the workspace after confirming that the `devfile.yaml` was committed and pushed successfully.

Verify Service Connectivity

Verify connectivity to the PostgreSQL service.

```
timeout 3 bash -c "echo > /dev/tcp/$POSTGRESQL_HOST/5432" \
&& echo "PostgreSQL connection successful" \
|| echo "PostgreSQL connection failed"
```

Verify that the command reports:

```
PostgreSQL connection successful
```



If connectivity fails, verify that you successfully completed the previous labs where PostgreSQL and Kafka were deployed.

Lab 11: Application Deployment

In this lab, you'll run the ETX App Base application in Quarkus development mode and verify that it can connect to the PostgreSQL database and Kafka cluster deployed earlier in the training.

Quarkus development mode provides a rapid development experience with live reload, allowing application changes to be reflected immediately without restarting the application.

By the end of this lab, you will have:

- Started the ETX App Base application in Quarkus development mode
- Verified connectivity to PostgreSQL and Kafka
- Accessed the running application from Red Hat Dev Spaces
- Learned how to manage the lifecycle of a Dev Spaces workspace

Run the Application in Quarkus Development Mode

When you open a terminal in a Dev Spaces workspace, you're automatically placed in the project directory.

First, verify your current working directory.

```
pwd
```

Verify that the command returns:

```
/projects/etx_app_base_app
```

Next, start Quarkus development mode.

```
./mvnw compile quarkus:dev
```

Quarkus uses the environment variables defined in your `devfile.yaml` to connect to the PostgreSQL database and Kafka cluster deployed in previous labs.

Wait for the application to finish starting. You should see output similar to the following.

```
Listening for transport dt_socket at address: 5005
__  ____  __  _____   ___  __ ____  ______
 --/ __ \/ / / / _ | / _ \/ //_/ / / / __/
 -/ /_/ / /_/ / __ |/ , _/ ,< / /_/ /\ \
--\___\_\____/_/ |_/_/|_/_/|_|\____/___/

INFO  [io.sma.rea.mes.kafka] SRMSG18229: Configured topics for channel 'emails-in':
[intake]
INFO  [io.sma.rea.mes.kafka] SRMSG18258: Kafka producer kafka-producer-emails-out,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.sma.rea.mes.kafka] SRMSG18257: Kafka consumer kafka-consumer-emails-in,
connected to Kafka brokers 'parasol-kafka-kafka-bootstrap.etx-app-
dev.svc.cluster.local:9092'
INFO  [io.quarkus] parasol-reimagined 1.0.0-SNAPSHOT on JVM (powered by Quarkus
3.17.5) started in 7.298s. Listening on: http://0.0.0.0:8080
INFO  [io.quarkus] Profile dev activated. Live Coding activated.

--
Tests paused
Press [e] to edit command line args, [r] to resume testing, [h] for more options>
```

Verify the following indicators of a successful startup:

- **Debug port available** (`5005`)
- **Kafka producer and consumer connected**
- **Application listening on port** `8080`
- **Live Coding activated**

Do not continue until Quarkus starts successfully and reports that Live Coding is activated.

Access the Running Application

After Quarkus starts, Red Hat Dev Spaces displays a notification in the lower-right corner with a link to the application endpoint.

Click **Open in New Tab** or **Open in Preview** to open the application.

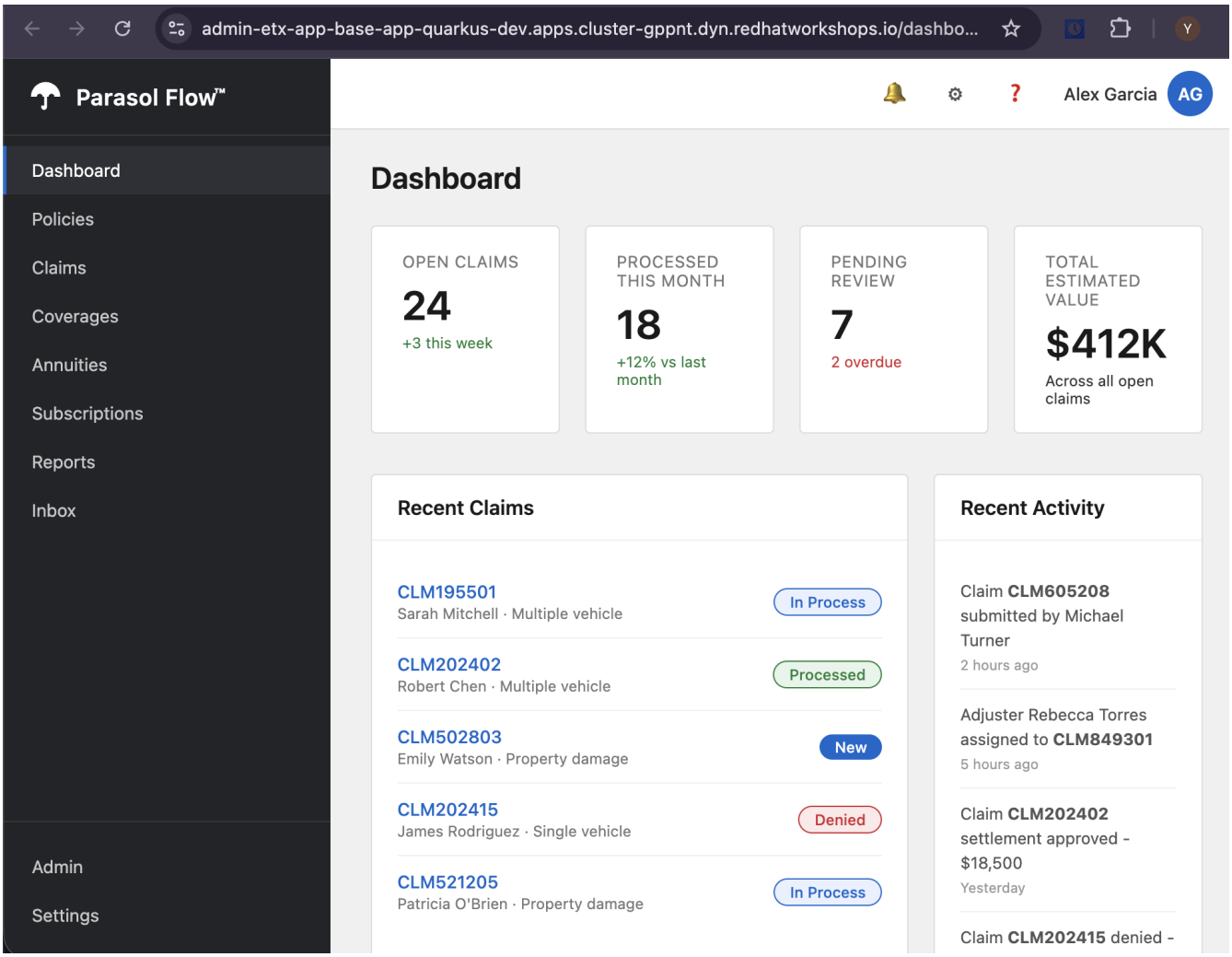


Figure 46. Application Endpoint Notification(Click the image to view it in full size in a new browser tab)

If you dismiss the notification, stop Quarkus by pressing `Ctrl` + `C`, then restart it using `./mvnw compile quarkus:dev`. The notification will appear again.

The application URL follows this format:

```
https://<username>-<project-name>-quarkus-dev.apps.<cluster-domain>/
```

For example:

```
https://admin-etx-app-base-app-quarkus-dev.apps.cluster-
```

```
grsml.dynamic2.redhatworkshops.io/
```

You can also locate the application URL in the **Endpoints** view within the Dev Spaces IDE.

Verify External Service Connectivity

The application automatically connects to the shared platform services using the environment variables defined in the Devfile.

Verify that the application is using the following services:

- **PostgreSQL**
  - Host: `parasol-db.etx-app-dev.svc.cluster.local`
  - Port: `5432`
  - Database: `parasol`
- **Apache Kafka**
  - Bootstrap server: `parasol-kafka-kafka-bootstrap.etx-app-dev.svc.cluster.local:9092`
  - Topic: `intake`



These shared services were deployed earlier in the training and are available to all development workspaces.

Manage the Workspace Lifecycle

Red Hat Dev Spaces allows you to stop, start, and delete workspaces while preserving your development environment when appropriate.

Stop a workspace

Stops the workspace pod while preserving the persistent workspace storage.

1. Open the Dev Spaces dashboard.
2. Select the workspace.
3. Click the options menu (⋮).
4. Select **Stop Workspace**.

Start a workspace

Restarts a previously stopped workspace while preserving your files, Maven cache, and Git repository.

1. Open the options menu (⋮).
2. Select **Start in Background**.

Delete a workspace

Deletes both the workspace pod and its associated persistent storage.

1. Open the options menu (⋮).
2. Select **Delete Workspace**.
3. Confirm the deletion.



Stop a workspace when you're finished developing but intend to continue later. Delete a workspace only when you no longer need the development environment.

⭐ Lab Summary

You now have the following components configured and ready:

- **Red Hat Dev Spaces** – Browser-based development environments running on OpenShift.
- **CheCluster** – Dev Spaces platform configured and accessible through the Dev Spaces dashboard.
- **GitLab Integration** – OAuth authentication configured for seamless source control access from developer workspaces.
- **Shared Workspace Secrets** – Common configuration and AI assistant credentials automatically injected into new workspaces.
- **Devfile** – A version-controlled development environment definition stored alongside the application source code.
- **ETX App Base Workspace** – A fully configured development workspace with Git, Maven, PostgreSQL, Kafka, and Quarkus tooling.
- **Quarkus Development Mode** – The application running with live reload and connected to the shared PostgreSQL database and Kafka cluster.

Your development environment now provides:

- **Source Control** – GitLab repositories accessible directly from Dev Spaces.
- **Application Runtime** – Quarkus development mode with hot reload.
- **Database Connectivity** – PostgreSQL available through `parasol-db:5432`.
- **Messaging Platform** – Apache Kafka available through `parasol-kafka-kafka-bootstrap:9092`.
- **Cloud-native Development** – A consistent, reproducible development environment defined by the project's `devfile.yaml`.

Knowledge Check

<iframe type="text/javascript" src="_attachments/questions.html" style="width: 100%; max-width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

Summary and What’s next?

Congratulations! You have successfully established the foundation of your Software Factory.

In this module, you learned how organizations standardize software delivery using a Software Factory, explored Git’s role in enabling collaborative development and GitOps workflows, prepared the required platform services, and configured a cloud-native development environment using OpenShift Dev Spaces and GitLab.

You now have everything required to begin building, deploying, and managing applications on the platform.

In the next modules, you’ll use this environment to implement automated software delivery pipelines, GitOps workflows, developer self-service capabilities, and trusted software supply chain practices.